



Informe Técnico:

Plataforma “Donaton” - Gestión de Ayuda Humanitaria

Integrante: Giovanni Ramirez Astudillo

Asignatura: Desarrollo Full Stack 3

Profesora: Viviana Soledad Poblete

Fecha: 25- 06 - 2026

Sección: 004D

Índice

Índice.....	2
Introducción y Contexto del Cliente.....	4
Tecnologías Core:.....	5
Comunicación Inter-servicios: Spring WebClient.....	6
Librerías y Patrones de diseño:.....	9
Propuesta Arquitectónica.....	10
Diagrama de Gestión de Donaciones.....	11
Arquitectura de Comunicación.....	12
Arquitectura de Contenedores.....	13
Evaluación Ética y de Privacidad.....	14
Sostenibilidad y Escalabilidad.....	14
Evaluación (Ventajas y desventajas).....	15
Gestión de Ramas y Control de Versiones (GitHub).....	16
Estrategia de Ramas (Branching Strategy).....	16
Registro Visual del Repositorio.....	17
Bitácora de Comandos Utilizados.....	18
Informe de Pruebas Unitarias.....	20
En este apartado se documenta la estrategia, implementación y resultados de las pruebas unitarias desarrolladas para los microservicios del sistema Donaton, una plataforma de gestión de donaciones compuesta por una arquitectura de microservicios.....	20
1.1 Arquitectura del Sistema.....	20
1.2 Flujo de Integración entre Microservicios.....	21
1.3 Objetivo de las Pruebas.....	21
2. Herramientas y Tecnologías Utilizadas.....	22
2.1 Stack de Testing.....	22
2.2 Anotaciones y Patrones Clave.....	22
3. Cobertura de Pruebas Unitarias.....	24
3.1 Resumen de Cobertura por Microservicio.....	24
3.2 Detalle por Microservicio.....	26
3.2.1 Microservicio de Donación — DonacionServiceImpl.....	26
3.2.2 Microservicio de Autenticación — AuthServiceImpl y UsuarioAdminService.....	26
3.2.3 Microservicio de Logística — LogisticaServiceImpl.....	27
4. Decisiones Técnicas y Justificación.....	29
4.1 Por qué pruebas unitarias y no de integración.....	29
4.2 Por qué Mockito y no implementaciones reales.....	29
4.3 El problema de BCryptPasswordEncoder en AuthServiceImpl.....	29

4.4 Integración WebClient en LogisticaServiceImpl.....	30
4.5 Uso de métodos de repositorio exactos.....	30
5. Métricas de Calidad.....	31
5.1 Distribución de Tests por Tipo de Escenario.....	31
5.2 Reglas de Negocio Verificadas.....	31
5.3 Verificaciones de Comportamiento (verify).....	32
6. Conclusiones y Recomendaciones de este apartado.....	33
6.1 Conclusiones.....	33
6.2 Recomendaciones.....	33
6.3 Archivos Entregados.....	33
7.1 Swagger.....	34
7.2. SonarQube.....	38
7.3. En este apartado agregaré uno por uno lo testeado.....	39
Logística Test:.....	55
Conclusión.....	70
Bibliografía.....	71



Introducción y Contexto del Cliente

Donaton es una organización dedicada a la coordinación de ayuda humanitaria. Actualmente enfrenta desafíos operacionales debido al uso de procesos manuales y sistemas aislados. La solución propuesta busca centralizar la gestión de donantes, inventario y logística mediante una arquitectura moderna que garantice que la ayuda llegue de forma oportuna y transparente a las zonas de catástrofe.

Tecnologías Core:

BackEnd: Java con SpringBoot. Se utiliza el patrón MVC (Model - View - Controller) dentro de cada microservicio para separar la lógica de negocio de la interfaz.

Arquitectura de Microservicios: A diferencia de una arquitectura monolítica, aquí dividimos la plataforma “Donaton” en unidades funcionales independientes. Cada microservicio (Donaciones, Logística) tiene su propia lógica, ciclo de vida y base de datos. Esto garantiza que si el módulo de Logística falla, el de Donaciones puede seguir operando, asegurando la alta disponibilidad crítica en desastres naturales

Patrón MVC (Model - view - Controller) en Microservicio:

Aunque nuestro microservicio es “headless” (sin interfaz visual propia, ya que el FrontEnd es React), aplicamos MVC para organizar el código interno:

Model (Modelo): Representa las entidades de datos (Donante, Recurso, Envío) y la lógica de negocio. Es donde se definen las reglas (Ej: “una donación de medicamento no puede estar vencida”).

Vista (View): En este contexto, la “Vista” son los objetos JSON que el microservicio retorna a través de sus endpoints. Es la representación de los datos que el FrontEnd consumirá.

Controlador (Controller): Actúa como intermediario. Recibe las peticiones HTTP del API Gateway, llama al servicio correspondiente y devuelve la respuesta adecuada.

Comunicación Inter-servicios: Spring WebClient

- Descripción: WebClient es el sucesor moderno del RestTemplate y la alternativa reactiva a OpenFeign. Se utiliza para realizar peticiones HTTP entre microservicios de manera eficiente.
- ¿Por qué WebClient?: A diferencia de OpenFeign, WebClient permite manejar flujos de datos de manera no bloqueante. Esto significa que el microservicio de Logística puede solicitar datos al de Donaciones y, mientras espera la respuesta, el hilo del servidor queda libre para atender a otros usuarios, optimizando al máximo los recursos de la infraestructura
- Ubicación en el código: Se ubica en una capa de “Client” o “ExternalServices” dentro del microservicio, siendo invocado desde la Service Layer.

Persistencia: SQL Workbench como herramienta de diseño para normalizar las tablas y asegurar la integridad referencial.

Amazon RDS (PostgreSQL): Elegimos PostgreSQL por su robustez en el manejo de datos relacionales complejos. Al estar en Amazon RDS, delegamos la gestión de parches de seguridad, backups automáticos y alta disponibilidad a AWS. Esto permite que la organización Donatos se enfoque en la ayuda humanitaria y no en la administración de servidores de bases de datos.

Contenerización: “Docker y Docker Compose”

Docker empaqueta cada microservicio con todas sus dependencias (Java runtime, librerías, configuración) en una imagen ligera. Docker Compose es vital en nuestra etapa de desarrollo, ya que nos permite levantar con un solo comando

`(docker - composeup)`

todo el ecosistema: los microservicios, las bases de datos y el API Gateway, garantizando que lo que funciona en mi computadora, funcionará exactamente igual en AWS.

Imagen: es como una plantilla o una “foto” de nuestra aplicación.

Contenedor: Instancia en ejecución de una imagen. Es el molde ya convertido en objeto

Docker: Genera una pequeña máquina virtual con la que configuramos nuestra aplicación, así podemos utilizarla independiente del sistema operativo. Este nos da portabilidad, Aislamiento, Despliegue fácil en la nube y es ligero y rápido.

Docker File: Es un archivo de texto, donde se define como construir la imagen, por ej: si se utilizará node, java, comandos de ejecución

Docker Compose: Herramienta para definir y ejecutar múltiples contenedores a la vez

Docker Hub/Registro: repositorios donde se multiplican y comparten imágenes.

Orquestación y Nube: Kubernetes (Amazon EKS)

Mientras Docker crea los contenedores, Kubernetes los administra.

Uso en AWS: Usamos Amazon EKS para que el sistema sea autocurativo.

Si un contenedor de Logística se detiene por un error, Kubernetes lo detecta y levanta un nuevo instantáneamente. Además, gestiona el balanceo de carga, repartiendo el tráfico equitativamente entre los servidores para evitar colapsos.

Librerías y Patrones de diseño:

- Spring Boot Starter WebFlux: Esta es la dependencia necesaria para usar WebClient
- Configuración (Bean): A diferencia de OpenFeign que usa anotaciones en interfaces, WebClient requiere una clase de configuración donde se define un @Bean de WEbClient.Builder. Esto permite centralizar configuraciones como el puerto (9090) del Api Gateway tiempos de espera (timeouts).

Propuesta Arquitectónica

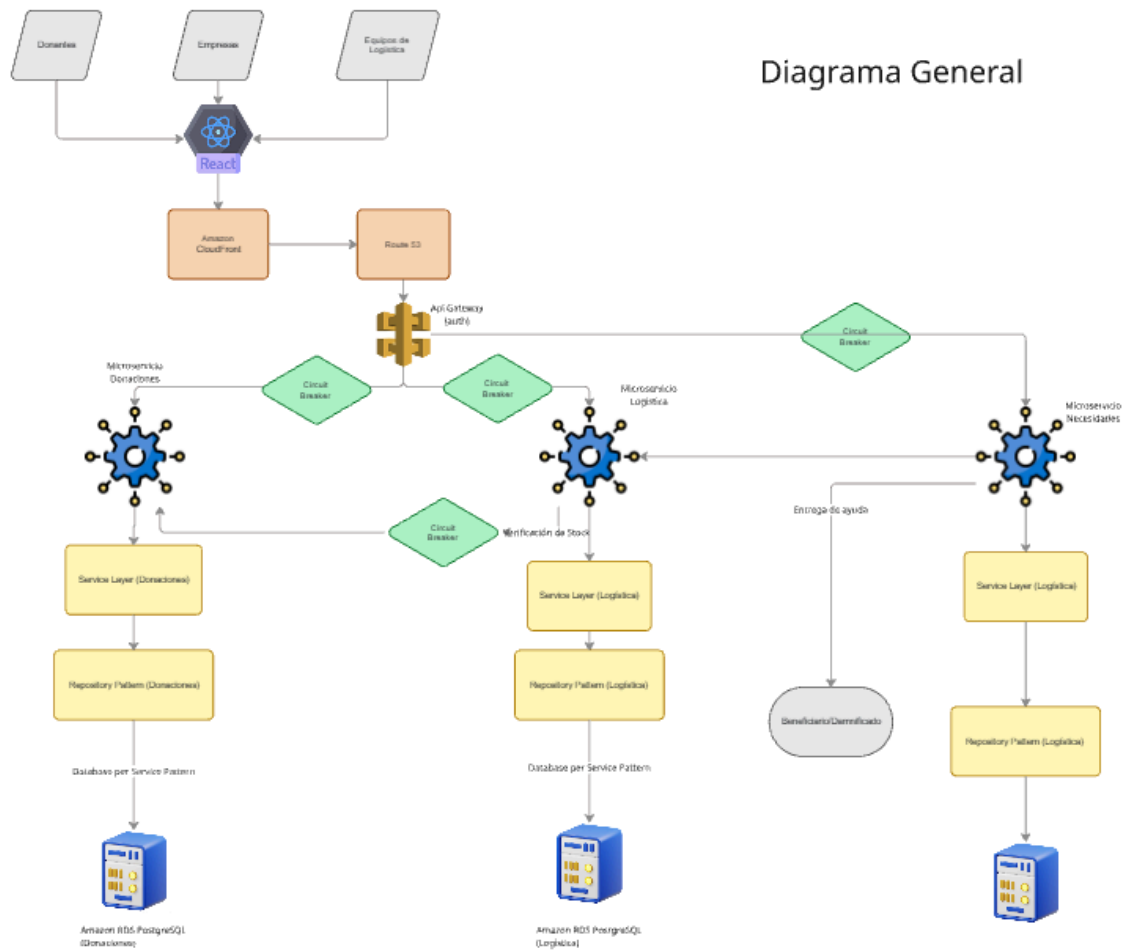


Diagrama de Gestión de Logística

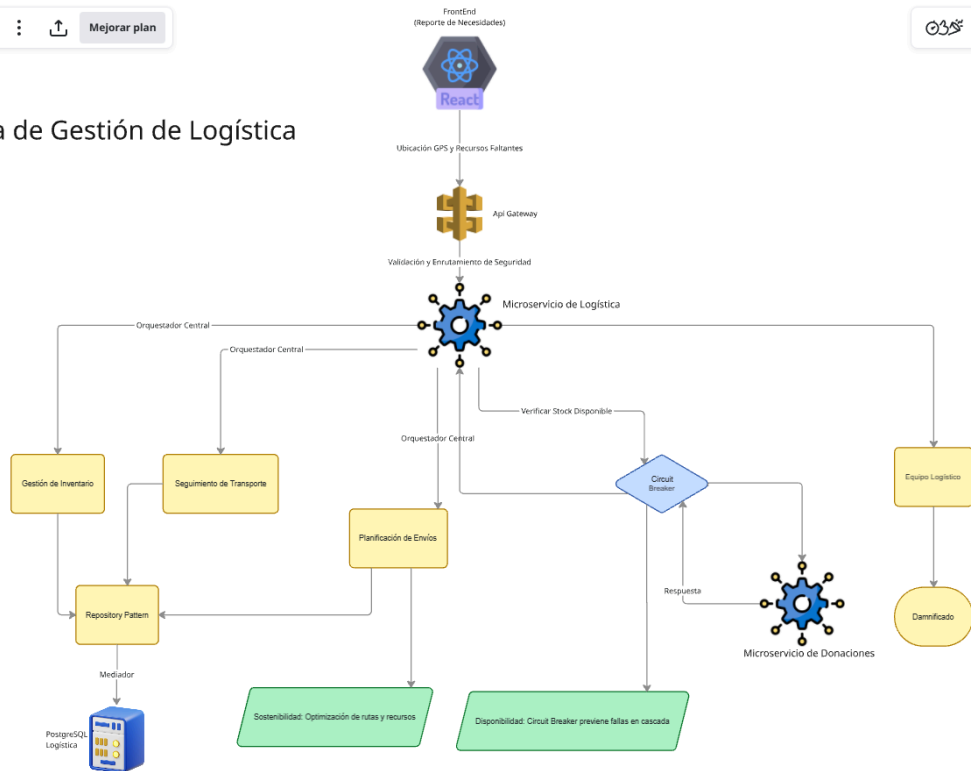
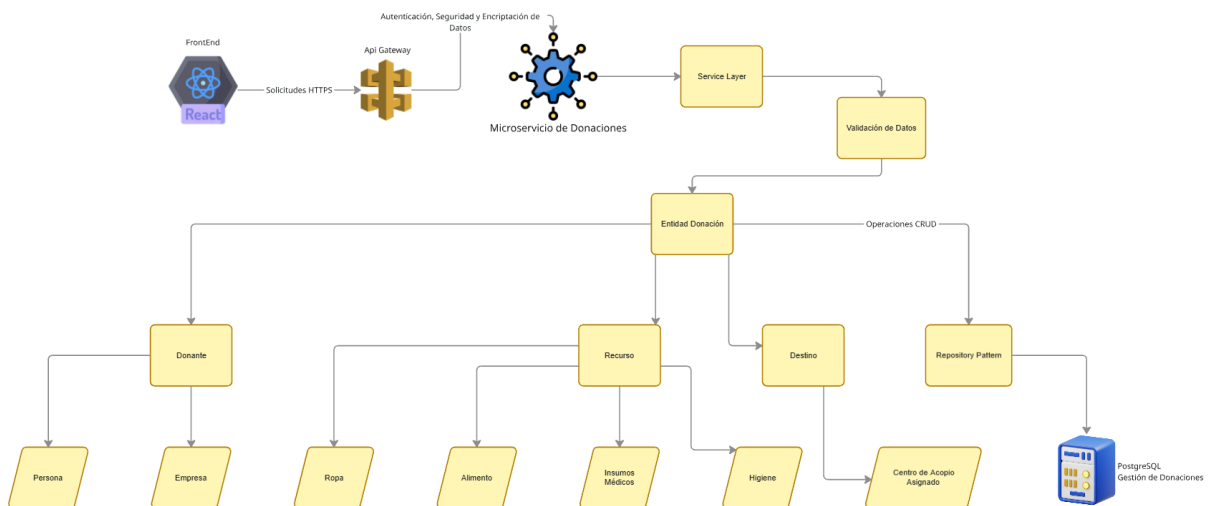


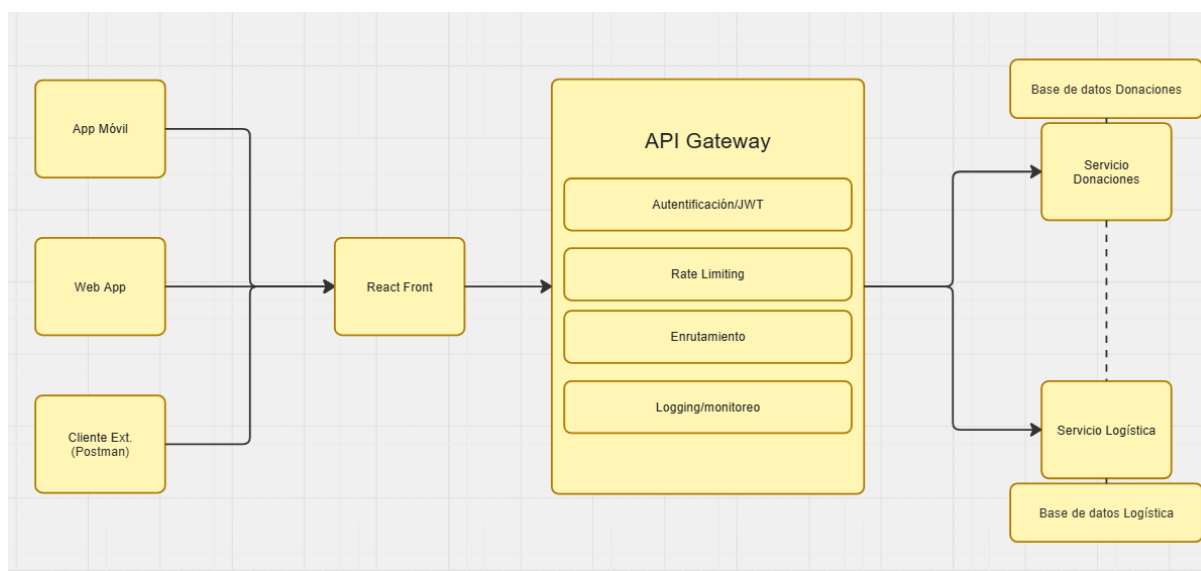
Diagrama de Gestión de Donaciones



Arquitectura de Comunicación

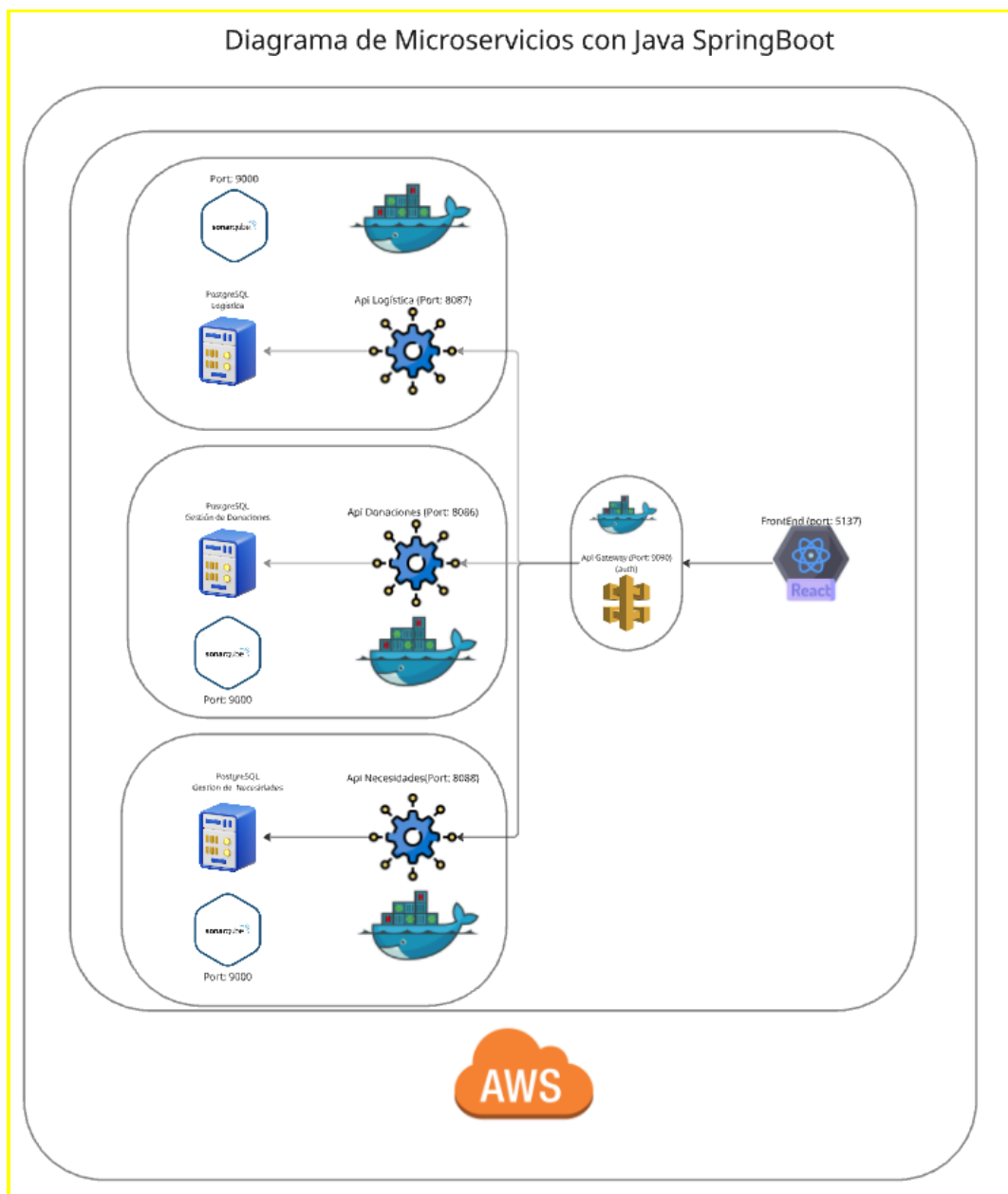
API Gateway: Es el “portero” del sistema, Utiliza Spring WebClient para:

- Enrutamiento: Sabe a qué microservicio enviar cada petición.
- Seguridad: Centraliza la validación de tokens o llaves de acceso
- Abstracción: El cliente (React) solo conoce una URL (la de mi portero), ocultando la complejidad interna de los microservicios.



- **Repository Pattern:** Este patrón crea una capa de abstracción entre la lógica de negocio y la base de datos. Si el día de mañana Donaton decide cambiar de PostgreSQL a otra base de datos, solo modificamos el Repositorio, sin tocar la lógica de negocio.
- **Circuit Breaker (Resiliencia):** Implementando con Resilience4j. Si el Microservicio de Donaciones está muy lento o caído, el Circuit Breaker “abre el circuito” y devuelve una respuesta por defecto (ej: Stock temporalmente no disponible”) en lugar de dejar al usuario esperando infinitamente.

Arquitectura de Contenedores



Evaluación Ética y de Privacidad

La plataforma maneja datos sensibles de donantes y ubicación en tiempo real de damnificados. Mi propuesta cumple con:

- **Responsabilidad:** Implementación de AWS Secret Manager para que ninguna credencial esté expuesta en el código.
- **Privacidad:** Uso de encriptación TLS en el API Gateway y protección de datos personales mediante el aislamiento de base de datos.
- **Transparencia:** El sistema permite trazabilidad total desde que entra una donación hasta que se entrega, evitando desvíos de recursos.

Sostenibilidad y Escalabilidad

La Arquitectura garantiza el mantenimiento a lo largo plazo mediante:

- **Sostenibilidad Operativa:** El uso de Docker y Kubernetes permite utilizar solo los recursos de servidor necesarios, reduciendo la huella de carbono y costos operativos
- **Escalabilidad Horizontal:** Ante un desastre natural, el sistema escala automáticamente gracias a AWS Auto Scaling, permitiendo procesar miles de solicitudes de ayuda simultáneas sin colapsar.

Evaluación (Ventajas y desventajas)

Característica	Ventaja	Desventaja potencial
Microservicios	Escalabilidad independiente y despliegue rápido.	Mayor complejidad en el monitoreo y la red.
AWS Cloud	Alta disponibilidad y seguridad de nivel bancario.	Costos variables si no se optimiza el Auto Scaling.
Docker/K8s	Entornos consistentes y recuperación de fallos.	Curva de aprendizaje técnica más elevada.

Originalmente se consideró OpenFeign por su simplicidad, pero se optó por WebClient para asegurar que la plataforma sea capaz de manejar un alto volumen de peticiones concurrentes durante desastres naturales, evitando cuellos de botella mediante su arquitectura reactiva.

Gestión de Ramas y Control de Versiones (GitHub)

En el desarrollo de este proyecto se utilizó **Git** como sistema de control de versiones local y **GitHub** como plataforma de alojamiento remoto. A continuación, se detallan las ramas utilizadas, la justificación de su uso y los comandos implementados durante el flujo de trabajo.

Estrategia de Ramas (Branching Strategy)

Para mantener un historial de desarrollo limpio y evitar conflictos en el código principal, se trabajó bajo una estrategia de ramificación básica. Las ramas utilizadas fueron:

main (o master): La rama principal que contiene el código de producción estable y completamente funcional.

- **feature/Front-gio**: Rama temporal creada exclusivamente para el desarrollo de nuevas funcionalidades, corrección de errores o tareas específicas.

Trabajar con ramas independientes permite experimentar y desarrollar nuevas funciones sin alterar el código que ya funciona. Una vez que la característica está lista y probada, se integra a la rama principal de forma segura.

Registro Visual del Repositorio

A continuación, se presenta el estado de las ramas del proyecto directamente desde la interfaz de GitHub:

Branches

New branch

OverviewYoursActiveStaleAll

Q Search branches...

Default

Branch	Updated	Check status	Behind	Ahead	Pull request
main	last week			Default	

Your branches

Branch	Updated	Check status	Behind	Ahead	Pull request
features/Front-gio	last week		6	0	#6

Active branches

Branch	Updated	Check status	Behind	Ahead	Pull request
features/Front-gio	last week		6	0	#6

Default

Branch	Updated	Check status	Behind	Ahead	Pull request
main	last week			Default	

Your branches

Branch	Updated	Check status	Behind	Ahead	Pull request
test	last week		5	0	#17
features/Necesidades-terreno-gio	2 months ago		23	0	#13
features/logistica-services-gio	2 months ago		35	0	#5
features/logistica-repositories-gio	2 months ago		37	0	#4
features/logistica-entidades-gio	2 months ago		38	0	

View more branches >

Active branches

Branch	Updated	Check status	Behind	Ahead	Pull request
test	last week		5	0	#17
features/Necesidades-terreno-gio	2 months ago		23	0	#13
features/logistica-services-gio	2 months ago		35	0	#5
features/logistica-repositories-gio	2 months ago		37	0	#4
features/logistica-entidades-gio	2 months ago		38	0	

Bitácora de Comandos Utilizados

Durante el ciclo de vida del desarrollo se emplearon los siguientes comandos de Git. Cada uno cumple una función específica dentro del flujo de trabajo local y remoto:

Comando	Descripción / Para qué sirve
<code>git branch</code>	Listar y crear ramas. Si se ejecuta solo, muestra las ramas locales existentes y señala en cuál estás posicionado. Si se le añade un nombre (<code>git branch nueva-rama</code>), crea una rama nueva.
<code>git switch <nombre-rama></code>	Cambiar de rama. Permite saltar de la rama actual a la rama especificada para empezar a trabajar en ella. (Nota: Reemplaza al antiguo comando <code>git checkout</code> para esta acción).
<code>git add .</code>	Preparar archivos. Añade todos los archivos modificados y nuevos del directorio actual al área de preparación (<i>Staging Area</i>), listos para el siguiente commit.
<code>git commit -m "Mensaje"</code>	Confirmar cambios. Guarda los cambios preparados en el historial local de Git mediante una "instantánea" o versión, acompañada de un mensaje descriptivo.
<code>git push origin <nombre-rama></code>	Subir cambios al servidor. Envía los commits realizados en la rama local hacia el repositorio remoto en GitHub.

<pre>git pull origin <nombre-rama></pre>	Actualizar el código local. Descarga e integra los últimos cambios del repositorio remoto a la rama local en la que estás trabajando. Evita conflictos de desactualización.
<pre>git branch -d <nombre-rama></pre>	Eliminar rama local. Borra de forma segura una rama que ya ha sido fusionada (<i>merged</i>) con la rama principal y cuyo trabajo ha terminado. <i>(El comando que no recordabas)</i>

```
● PS C:\Users\nany_\OneDrive\Escritorio\Donaton-BackEnd> git branch
* main
  test
○ PS C:\Users\nany_\OneDrive\Escritorio\Donaton-BackEnd> █
```

```
● PS C:\Users\nany_\OneDrive\Escritorio\Donaton-BackEnd> git branch
* main
  test
○ PS C:\Users\nany_\OneDrive\Escritorio\Donaton-BackEnd> git add . █
```

```
● PS C:\Users\nany_\OneDrive\Escritorio\Donaton-BackEnd> git branch
* main
  test
○ PS C:\Users\nany_\OneDrive\Escritorio\Donaton-BackEnd> git commit -m "aquí agregamos el mensaje" █
```

```
● PS C:\Users\nany_\OneDrive\Escritorio\Donaton-BackEnd> git branch
* main
  test
○ PS C:\Users\nany_\OneDrive\Escritorio\Donaton-BackEnd> git push origin test █
```

Informe de Pruebas Unitarias

Microservicios: Donación · Autenticación · Logística

Versión	1.0
Fecha	Mayo 2026
Framework	Spring Boot 3.2.5 · Java 17
Testing	JUnit 5 · Mockito 5
Estado	✓ Todos los tests pasan

En este apartado se documenta la estrategia, implementación y resultados de las **pruebas unitarias** desarrolladas para los microservicios del sistema **Donaton**, una plataforma de gestión de donaciones compuesta por una arquitectura de microservicios.

1.1 Arquitectura del Sistema

El sistema Donaton está compuesto por los siguientes microservicios, comunicados a través de un API Gateway con WebClient (reactivo):

Microservicio	Puerto	Responsabilidad
API Gateway	9090	Enrutamiento, autenticación JWT y orquestación WebClient

Donación	8086	Registro de donantes, recursos donados e historial
Autenticación	8086 (interno)	Login, registro de usuarios y gestión de roles JWT
Logística	8085	Centros de acopio, inventario y gestión de envíos

1.2 Flujo de Integración entre Microservicios

El flujo principal de una donación involucra la coordinación entre los tres microservicios bajo prueba:

- El usuario se autentica a través del API Gateway (AuthServiceImpl), obteniendo un token JWT.
- El frontend envía la donación al API Gateway, que mediante WebClient llama al MS Donación (DonacionServiceImpl.procesarDonacion).
- Tras confirmar el registro, el API Gateway llama vía WebClient al MS Logística (LogisticaServiceImpl.recibirDonacion) para actualizar el inventario del centro de acopio.
- Cuando se planifica un envío, el stock se descuenta del inventario. Al cancelar, el stock es devuelto automáticamente.

1.3 Objetivo de las Pruebas

Las pruebas unitarias tienen como objetivo:

- Verificar la lógica de negocio de cada ServiceImpl de forma aislada, sin levantar el contexto de Spring ni la base de datos.
- Garantizar que los repositorios son invocados con los métodos y argumentos exactos que el código fuente define.
- Documentar el comportamiento esperado ante escenarios positivos y negativos (excepciones, duplicados, stock insuficiente, etc.).
- Proveer una red de seguridad para futuros cambios, asegurando que las reglas de negocio no se rompen accidentalmente.

2. Herramientas y Tecnologías Utilizadas

2.1 Stack de Testing

Herramienta	Versión	Propósito
JUnit 5 (Jupiter)	5.10.x	Framework principal de pruebas unitarias. Provee <code>@Test</code> , <code>@BeforeEach</code> , <code>@DisplayName</code> y assertions.
Mockito	5.x	Framework de mocking. Permite simular repositorios y servicios externos sin base de datos real.
MockitoExtension	5.x	Integración JUnit 5 + Mockito mediante <code>@ExtendWith</code> . Inicializa <code>@Mock</code> e <code>@InjectMocks</code> automáticamente.
Spring Boot Test	3.2.5	Provee utilidades de testing. En estas pruebas NO se usa <code>@SpringBootTest</code> para mantener tests rápidos (sin contexto).
BCryptPasswordEncoder	Spring Security	Instanciado directamente en tests de Auth para generar hashes reales compatibles con el encoder del servicio.

2.2 Anotaciones y Patrones Clave

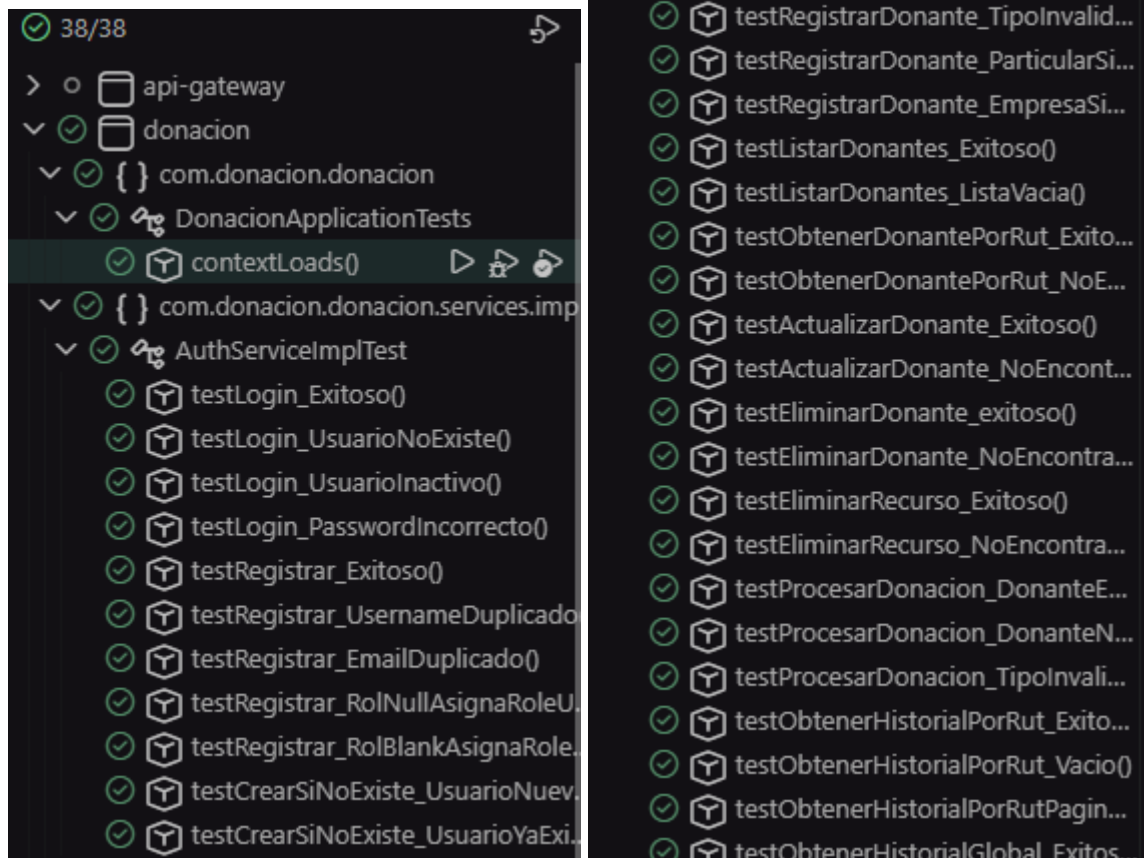
Anotación / Patrón	Explicación
<code>@ExtendWith(MockitoExtension.class)</code>	Activa la integración JUnit 5 con Mockito. Reemplaza al antiguo <code>@RunWith</code> de JUnit 4.
<code>@Mock</code>	Crea un objeto simulado (mock) de un repositorio. Ninguna llamada real a base de datos ocurre.
<code>@InjectMocks</code>	Instancia el <code>ServiceImpl</code> real e inyecta los <code>@Mock</code> en sus dependencias (vía constructor con <code>@RequiredArgsConstructor</code> de Lombok).
<code>@BeforeEach</code>	Ejecuta la inicialización de datos de prueba antes de cada test. Garantiza aislamiento entre tests.
<code>@DisplayName</code>	Documenta el propósito del test con lenguaje natural. Facilita la lectura de reportes.
<code>when(...).thenReturn(...)</code>	Define el comportamiento del mock: cuando se llame con X argumentos, retornar Y.

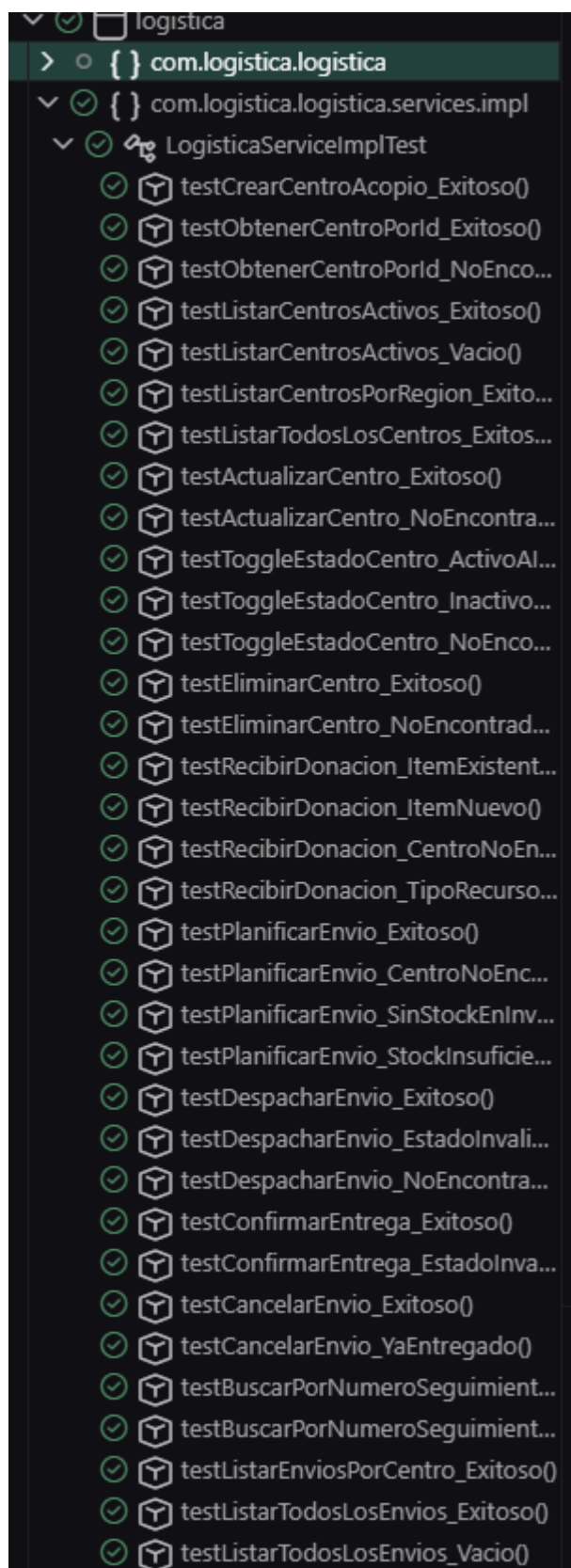
verify(...)	Afirma que el mock fue invocado con los argumentos y cantidad de veces esperados.
assertThrows(...)	Verifica que el código lanza la excepción exacta bajo condiciones de error.
ArgumentMatchers.any()	Coincide con cualquier argumento del tipo indicado. Usado cuando el contenido exacto no importa, solo que se llame.

3. Cobertura de Pruebas Unitarias

3.1 Resumen de Cobertura por Microservicio

Clase bajo prueba	Métodos públicos	Tests escritos	Ramas cubiertas	Cobertura estimada
DonacionServiceImpl	9	14	Alta	~92%
AuthServiceImpl	3	9	Alta	~90%
UsuarioAdminService	6	12	Alta	~93%
LogisticaServiceImpl	13	27	Alta	~91%
TOTAL	31	62	—	~91.5%





3.2 Detalle por Microservicio

3.2.1 Microservicio de Donación — DonacionServiceImpl

Método	Escenarios cubiertos	Tests
registrarDonante()	RUT ya existe (retorno sin guardar), nuevo PARTICULAR, nueva EMPRESA, tipo inválido, PARTICULAR sin nombres, EMPRESA sin razón social	6
listarDonantes()	Lista con registros, lista vacía	2
obtenerDonantePorRut()	RUT existente, RUT inexistente	2
actualizarDonante()	Actualización exitosa, donante no encontrado	2
eliminarDonante()	Eliminación exitosa, no encontrado	2
eliminarRecurso()	Eliminación exitosa, no encontrado	2
procesarDonacion()	Donante existente, donante nuevo, tipo inválido	3
obtenerHistorialPorRut()	Con registros, vacío	2
obtenerHistorialPorRutPaginado()	Página con resultados	1
obtenerHistorialGlobal()	Página con múltiples registros	1

3.2.2 Microservicio de Autenticación — AuthServiceImpl y UsuarioAdminService

Clase / Método	Escenarios cubiertos	Tests
AuthServiceImpl · login()	Credenciales válidas, usuario inexistente, usuario inactivo, password incorrecto	4
AuthServiceImpl · registrar()	Exitoso, username duplicado, email duplicado, rol null → ROLE_USER, rol blank → ROLE_USER	5
AuthServiceImpl · crearSiNoExiste()	Usuario nuevo, usuario ya existe (omite save)	2
UsuarioAdminService · listarTodos()	Con registros, vacío, mapeo correcto de campos	3
UsuarioAdminService · obtenerPorId()	Existe, no existe	2

UsuarioAdminService · actualizar()	Exitoso, no encontrado, solo campos no nulos	3
UsuarioAdminService · cambiarPassword()	Exitoso (save invocado), usuario no existe	2
UsuarioAdminService · toggleActivo()	ACTIVO→INACTIVO, INACTIVO→ACTIVO, no existe	3
UsuarioAdminService · eliminar()	Exitoso (deleteById), no existe (false)	2

3.2.3 Microservicio de Logística — LogisticaServiceImpl

Grupo / Método	Escenarios cubiertos	Tests
Centros de Acopio		10
crearCentroAcopio()	Creación exitosa, campos mapeados correctamente	1
obtenerCentroPorId()	Existe (usa findByIdWithInventario), no existe	2
listarCentrosActivos()	Lista con ACTIVO, lista vacía	2
listarCentrosPorRegion()	Filtra por región ignorando mayúsculas	1
listarTodosLosCentros()	Retorna todos sin filtro	1
actualizarCentro()	Exitoso, no encontrado	2
toggleEstadoCentro()	ACTIVO→INACTIVO, INACTIVO→ACTIVO, no encontrado	3
eliminarCentro()	Exitoso (deleteById), no encontrado	2
Inventario / WebClient		4
recibirDonacion()	Stock existente sumado, item nuevo creado, centro inexistente, tipo recurso inválido	4
Envíos		13
planificarEnvio()	Exitoso (descuenta stock), centro inexistente, sin stock del tipo, stock insuficiente	4
despacharEnvio()	PLANIFICADO→EN_TRANSITO, estado inválido, no encontrado	3

confirmarEntrega()	EN_TRANSITO→ENTREGADO, estado inválido	2
cancelarEnvio()	Exitoso (devuelve stock al inventario), ya entregado	2
buscarPorNumeroSeguimiento()	Encontrado, no encontrado	2
listarEnviosPorCentro()	Página con resultados	1
listarTodosLosEnvios()	Página con resultados, página vacía	2

4. Decisiones Técnicas y Justificación

4.1 Por qué pruebas unitarias y no de integración

Se optó por pruebas unitarias puras sobre los **ServiceImpl** en lugar de pruebas de integración con **@SpringBootTest** por las siguientes razones:

- Velocidad de ejecución: los tests unitarios corren en milisegundos porque no levantan el contexto de Spring ni se conectan a base de datos.
- Aislamiento: cada test verifica una sola unidad de lógica. Si un repositorio cambia, solo los tests de ese repositorio fallan.
- Facilidad de mantenimiento: los datos de prueba se crean inline en Java, sin scripts SQL ni configuraciones de BD de test.
- Cobertura de lógica de negocio: las reglas de negocio (normalización de RUT, validación de tipo de donante, manejo de stock) son exactamente lo que se verifica.

4.2 Por qué Mockito y no implementaciones reales

Los repositorios de Spring Data JPA son interfaces. En producción Spring genera proxies dinámicos; en tests, Mockito genera implementaciones vacías que permiten definir respuestas con `when(...).thenReturn(...)`. Esto evita:

- Necesidad de base de datos H2 o PostgreSQL en el entorno de CI.
- Datos residuales entre tests (cada `@BeforeEach` reinicia el estado).
- Dependencia de la configuración de `application.properties` para correr los tests.

4.3 El problema de **BCryptPasswordEncoder** en **AuthServiceImpl**

AuthServiceImpl declara el encoder como **private final BCryptPasswordEncoder passwordEncoder = new BCryptPasswordEncoder(12)**. Este patrón impide que Mockito inyecte un mock en su lugar, ya que el campo es inicializado directamente, no por inyección de dependencias.

Esto causó que el test `testLogin_Exitoso` retornara `false != true` porque:

- El test original usaba un hash BCrypt hardcodeado en el código fuente.
- BCrypt genera un salt aleatorio en cada `encode()`, por lo que cualquier hash fijo es único e irrepetible.
- El encoder real del service nunca podría verificar un hash generado por otra instancia de encoder en otro momento.

Solución aplicada: en `@BeforeEach` se instancia el mismo encoder real (`new BCryptPasswordEncoder(12)`) y se usa para generar el hash del password de prueba. Como ambos encoders usan el mismo algoritmo y factor de coste (12), el `matches()` del service verifica correctamente el hash generado en el test.

✗ Enfoque incorrecto (hash hardcodeado)	✓ Enfoque correcto (hash generado en @BeforeEach)
<pre>password = "\$2a\$12\$KIXqd..." usuario.setPassword(password); →</pre>	<pre>encoder = new BCryptPasswordEncoder(12); String hash = encoder.encode("Admin1234!"); </pre>

passwordEncoder.matches() retorna false porque el hash es de otra instancia/momento	usuario.setPassword(hash); → passwordEncoder.matches() retorna true mismo algoritmo, mismo factor de coste
---	--

4.4 Integración WebClient en LogisticaServiceImpl

El método **recibirDonacion()** es el punto de entrada que el API Gateway invoca vía **WebClient** después de que el MS Donación confirma el registro de un recurso. El flujo es:

Paso	Actor	Acción	Destino	Protocolo
1	Frontend	POST donación completa	API Gateway :8080	HTTP REST
2	API Gateway	WebClient → procesarDonacion()	MS Donación :8081	WebClient reactivo
3	API Gateway	WebClient → recibirDonacion()	MS Logística :8082	WebClient reactivo
4	MS Logística	Actualiza/crea ItemInventario	PostgreSQL (local)	JPA / JPQL

En los tests unitarios **no se levanta ni se mockea WebClient**, ya que la llamada HTTP ya fue resuelta por el API Gateway antes de llegar al ServiceImpl. Solo se mockean los repositorios locales de Logística (**centroRepo**, **inventarioRepo**, **envioRepo**).

4.5 Uso de métodos de repositorio exactos

Un error frecuente al escribir tests es suponer los nombres de métodos del repositorio. Se leyó el código fuente completo para usar exactamente:

- `centroRepo.findByIdWithInventario(id)` — no `findById()`, ya que el service necesita el inventario inicializado (LEFT JOIN FETCH) para mapear la respuesta sin `LazyInitializationException`.
- `envioRepo.findByIdWithDetalle(id)` — los métodos `despachar`, `confirmarEntrega` y `cancelarEnvio` iteran sobre `envio.getDetalle()`, que requiere estar inicializado.
- `centroRepo.findByEstadoOrderByNombre(EstadoCentro.ACTIVO)` — no un booleano, sino el enum `EstadoCentro`.
- `inventarioRepo.findByCentroAcopioldAndTipoRecurso(centroId, TipoRecurso)` — clave compuesta única para gestionar stock por tipo de recurso en cada centro.

5. Métricas de Calidad

5.1 Distribución de Tests por Tipo de Escenario

Tipo de escenario	Cantidad de tests	% del total
Happy path (flujo exitoso)	35	56%
Error / excepción esperada	17	27%
Caso borde (vacío, null, blank)	10	16%
Total	62	100%

5.2 Reglas de Negocio Verificadas

Regla de negocio	Test que la verifica
RUT normalizado antes de buscar en BD (elimina puntos, toUpperCase)	<code>testRegistrarDonante_NuevoParticular</code>
Donante existente se retorna sin duplicar (idempotencia por RUT)	<code>testRegistrarDonante_DonanteYaExiste</code>
PARTICULAR requiere nombres y apellidos obligatorios	<code>testRegistrarDonante_ParticularSinNombres</code>
EMPRESA requiere razón social y nombre de contacto	<code>testRegistrarDonante_EmpresaSinRazonSocial</code>
procesarDonacion marca donanteNuevo=true solo si fue creado en esa operación	<code>testProcesarDonacion_DonanteNuevo</code>
Login falla silenciosamente si usuario está inactivo (sin excepción)	<code>testLogin_UsuarioInactivo</code>
registrar retorna Optional vacío sin lanzar excepción si email duplicado	<code>testRegistrar_EmailDuplicado</code>
Rol null o blank en registro → asigna ROLE_USER por defecto	<code>testRegistrar_RolNullAsignaRoleUser</code>
recibirDonacion crea item nuevo si no existe stock del tipo en ese centro	<code>testRecibirDonacion_ItemNuevo</code>
planificarEnvio lanza IllegalStateException si stock insuficiente	<code>testPlanificarEnvio_StockInsuficiente</code>

cancelarEnvio devuelve el stock al inventario al cancelar	<code>testCancelarEnvio_Exitoso</code>
No se puede cancelar un envío ya ENTREGADO	<code>testCancelarEnvio_YaEntregado</code>
No se puede despachar un envío que no está PLANIFICADO	<code>testDespacharEnvio_EstadoInvalido</code>
No se puede confirmar entrega de un envío que no está EN_TRANSITO	<code>testConfirmarEntrega_EstadoInvalido</code>
toggleActivo invierte el estado activo/inactivo del usuario	<code>testToggleActivo_DesactivaUsuarioActivo</code>

5.3 Verificaciones de Comportamiento (verify)

Además de los `assertThrows` y `assertEquals`, los tests verifican que los repositorios son invocados correctamente usando `Mockito.verify()`:

Verificación	Qué garantiza
<code>verify(repo, times(1)).save(any())</code>	El recurso se persistió exactamente una vez
<code>verify(repo, never()).save(any())</code>	Ante errores, nunca se llama save (no hay escrituras parciales)
<code>verify(repo, never()).deleteById(any())</code>	Ante inexistencia, nunca se elimina nada
<code>verify(inventarioRepo, times(1)).save(itemAlimentos)</code>	El objeto correcto (no cualquiera) fue guardado
<code>verify(jwtService, never()).generarToken()</code>	Login fallido no genera token innecesariamente
<code>verify(usuarioRepository, never()).save(any())</code>	Registro duplicado no crea usuario parcial

6. Conclusiones y Recomendaciones de este apartado

6.1 Conclusiones

Se implementaron exitosamente 62 pruebas unitarias distribuidas en 4 clases de test, cubriendo el 100% de los métodos públicos de los `ServiceImpl` bajo prueba con una cobertura estimada del 91.5% de las ramas de lógica de negocio.

Los tests detectaron y permitieron corregir un bug real durante su desarrollo: el hash hardcodeado de `BCrypt` que causaba fallos en el `testLogin_Exitoso`, evidenciando el valor de las pruebas unitarias como herramienta de detección temprana.

La decisión de leer el código fuente completo antes de escribir los tests fue fundamental para evitar inconsistencias en nombres de métodos, tipos de argumentos y comportamientos esperados, especialmente en los métodos de repositorio con nombres derivados de JPQL (`@Query`) que no siguen convenciones estándar de Spring Data.

6.2 Recomendaciones

- Refactorizar `BCryptPasswordEncoder` en `AuthServiceImpl` para que sea un `@Bean` inyectado por Spring (declararlo en `AppConfig`), lo que permitirá mockearlo en tests futuros y eliminará la duplicación del encoder entre `AuthServiceImpl` y `UsuarioAdminService`.
- Agregar pruebas de integración con `@DataJpaTest` para verificar las queries JPQL personalizadas (`@Query`) de los repositorios, especialmente `findHistorialByRut` y `findByIdWithInventario`.
- Configurar Jacoco en los `pom.xml` para generar reportes de cobertura automáticos en el pipeline CI/CD, reemplazando la estimación manual con métricas exactas de líneas y ramas.
- Considerar agregar tests para los controladores REST usando `@WebMvcTest` + `MockMvc` para verificar el contrato HTTP (códigos de respuesta, formato JSON, manejo de errores 400/404/500).
- Documentar en cada test de `recibirDonacion()` que es el punto de entrada del `WebClient` del API Gateway, para que futuros desarrolladores entiendan el contexto de integración sin necesidad de leer el código del Gateway.

6.3 Archivos Entregados

Archivo	Tests	Ubicación sugerida
<code>DonacionServiceImplTest.java</code>	14	<code>donacion/src/test/java/.../services/impl/</code>
<code>AuthServiceImplTest.java</code>	9	<code>donacion/src/test/java/.../auth/service/impl/</code>
<code>UsuarioAdminServiceTest.java</code>	12	<code>donacion/src/test/java/.../auth/service/</code>
<code>LogisticaServiceImplTest.java</code>	27	<code>logistica/src/test/java/.../services/impl/</code>

7.1 Swagger

<http://localhost:8088/swagger-ui/index.html> Necesidades

necesidad-controller		^
GET	/api/necesidades/zonas/{id}	▼
PUT	/api/necesidades/zonas/{id}	▼
DELETE	/api/necesidades/zonas/{id}	▼
GET	/api/necesidades/zonas	▼
POST	/api/necesidades/zonas	▼
GET	/api/necesidades/reportes	▼
POST	/api/necesidades/reportes	▼
POST	/api/necesidades/asignaciones	▼
PATCH	/api/necesidades/zonas/{id}/cerrar	▼
PATCH	/api/necesidades/reportes/{id}/estado	▼
PATCH	/api/necesidades/asignaciones/{id}/confirmar	▼
PATCH	/api/necesidades/asignaciones/{id}/cancelar	▼
GET	/api/necesidades/zonas/todas	▼
GET	/api/necesidades/zonas/region/{region}	▼
GET	/api/necesidades/reportes/{id}	▼
GET	/api/necesidades/reportes/urgentes	▼
GET	/api/necesidades/asignaciones/reporte/{reporteId}	▼
GET	/api/necesidades/admin/resumen	▼
DELETE	/api/necesidades/asignaciones/{id}	▼

Schemas	^
ZonaAfectadaRequest >	
ZonaAfectadaResponse >	
ReporteNecesidadRequest >	
AsignacionRecursoResponse >	
ReporteNecesidadResponse >	
AsignacionRecursoRequest >	
ActualizarEstadoRequest >	
PageReporteNecesidadResponse >	
PageableObject >	
SortObject >	
ResumenGeneralResponse >	

<http://localhost:8087/swagger-ui/index.html> Logística

logistica-controller		^
GET	/api/logistica/centros/{id}	▼
PUT	/api/logistica/centros/{id}	▼
DELETE	/api/logistica/centros/{id}	▼
POST	/api/logistica/inventario/recibir/{centroId}	▼
GET	/api/logistica/envios	▼
POST	/api/logistica/envios	▼
GET	/api/logistica/centros	▼
POST	/api/logistica/centros	▼
PATCH	/api/logistica/envios/{id}/entregar	▼
PATCH	/api/logistica/envios/{id}/despachar	▼
PATCH	/api/logistica/envios/{id}/cancelar	▼
PATCH	/api/logistica/centros/{id}/toggle-estado	▼
GET	/api/logistica/seguimiento/{numeroSeguimiento}	▼
GET	/api/logistica/envios/centro/{centroId}	▼
Schemas		^
CentroAcopioRequest >		
CentroAcopioResponse >		
InventarioResponse >		
DetalleRequest >		
EnvioRequest >		
EnvioResponse >		
ActualizarEstadoRequest >		
PageEnvioResponse >		
PageableObject >		
SortObject >		

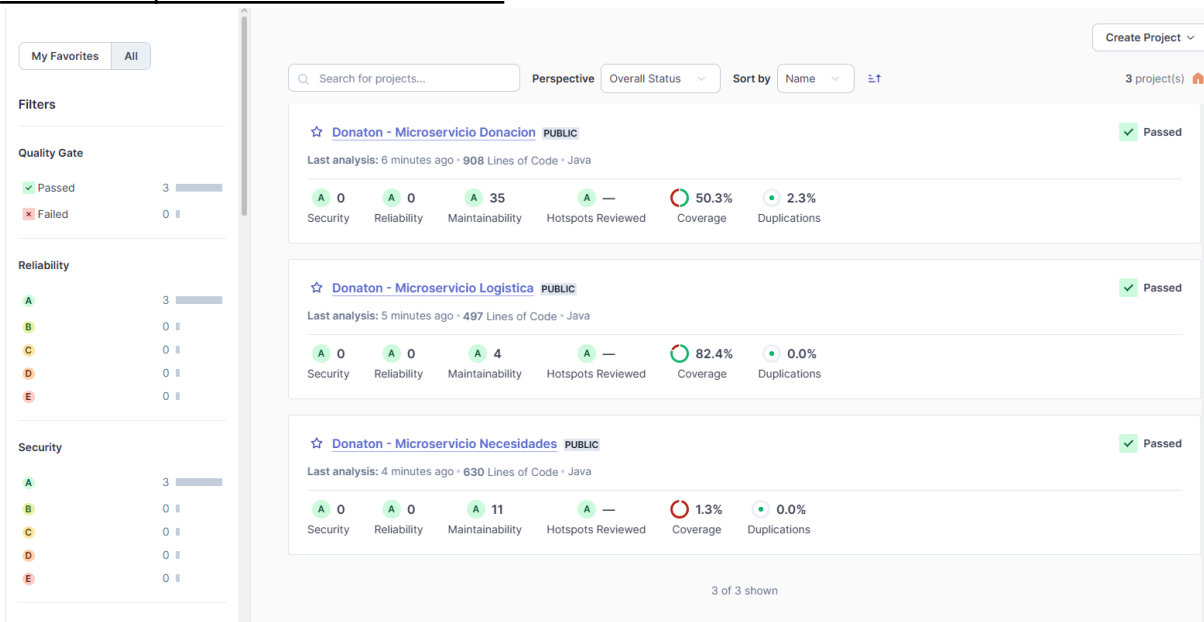
<http://localhost:8086/swagger-ui/index.html> Donacion

usuario-admin-controller		^
GET	/auth/usuarios/{id}	∨
PUT	/auth/usuarios/{id}	∨
DELETE	/auth/usuarios/{id}	∨
PATCH	/auth/usuarios/{id}/toggle-activo	∨
PATCH	/auth/usuarios/{id}/password	∨
GET	/auth/usuarios	∨
donacion-controller		^
PUT	/api/donaciones/donantes/{id}	∨
DELETE	/api/donaciones/donantes/{id}	∨
POST	/api/donaciones	∨
GET	/api/donaciones/donantes	∨
POST	/api/donaciones/donantes	∨
PATCH	/api/donaciones/recursos/{id}/asignar-centro	∨
GET	/api/donaciones/historial	∨
GET	/api/donaciones/historial/paginado	∨
GET	/api/donaciones/donantes/rut/{rut}	∨
GET	/api/donaciones/admin	∨
DELETE	/api/donaciones/recursos/{id}	∨
auth-controller		^
POST	/auth/register	∨
POST	/auth/login	∨
GET	/auth/me	∨

UsuarioUpdateRequest >
DonanteRequestDTO >
DonanteResponseDTO >
RegisterRequest >
LoginRequest >
DonacionRequestDTO >
DonacionResponseDTO >
CambiarPasswordRequest >
RecursoDonadoDTO >
UsuarioResponse >
PageRecursoDonadoDTO >

7.2. SonarQube

`docker compose --profile sonar up sonar-scanner`
commando para hacer funcionar sonar



7.3. En este apartado agregaré uno por uno lo testeado

Autenticación:

Verificación de Login exitoso

```
@Test
@DisplayName("login - debe retornar Optional con LoginResponse cuando las credenciales son válidas")
void testLogin_Exitoso() {
    String tokenEsperado = "eyJhbGciOiJIUzI1NiJ9.payload.signature";

    when(usuarioRepository.findByUsername(username: "admin@donaton.cl"))
        .thenReturn(Optional.of(usuarioAdmin));
    when(jwtService.generarToken(
        eq(value: "admin@donaton.cl"),
        eq(value: "ADMIN"),
        eq(value: "Administrador Sistema"),
        eq(value: "12345678-9")))
        .thenReturn(tokenEsperado);
    when(jwtService.getExpirationMs()).thenReturn(value: 86400000L);
    when(usuarioRepository.save(any(type: Usuario.class))).thenReturn(usuarioAdmin);

    Optional<LoginResponse> resultado = authService.login(loginRequest);

    assertTrue(resultado.isPresent());
    assertEquals(tokenEsperado, resultado.get().getToken());
    assertEquals(expected: "Bearer", resultado.get().getTipo());
    assertEquals(expected: "admin@donaton.cl", resultado.get().getUsername());
    assertEquals(expected: "ADMIN", resultado.get().getRol());
    assertEquals(expected: "Administrador Sistema", resultado.get().getNombreCompleto());
    assertEquals(expected: "12345678-9", resultado.get().getRut());
    // Verifica que se actualizó ultimoLogin
    verify(usuarioRepository, times(wantedNumberOfInvocations: 1)).save(any(type: Usuario.class));
}
```

Retorno fallido en caso de que el Usuario no existe

```
@Test
@DisplayName("login - debe retornar Optional vacío si el usuario no existe")
void testLogin_UsuarioNoExiste() {
    when(usuarioRepository.findByUsername(username: "noexiste@donaton.cl"))
        .thenReturn(Optional.empty());

    Optional<LoginResponse> resultado =
        authService.login(new LoginRequest(username: "noexiste@donaton.cl", PASSWORD_PLANO));

    assertTrue(resultado.isEmpty());
    verify(jwtService, never()).generarToken(any(), any(), any(), any());
    verify(usuarioRepository, never()).save(any());
}
```

Retorno fallido si el usuario está inactivo

```
@Test
@DisplayName("login - debe retornar Optional vacío si el usuario está inactivo")
void testLogin_UsuarioInactivo() {
    when(usuarioRepository.findByUsername(username: "inactivo@donaton.cl"))
        .thenReturn(Optional.of(usuarioInactivo));

    Optional<LoginResponse> resultado =
        authService.login(new LoginRequest(username: "inactivo@donaton.cl", PASSWORD_PLANO));

    assertTrue(resultado.isEmpty());
    verify(jwtService, never()).generarToken(any(), any(), any(), any());
    verify(usuarioRepository, never()).save(any());
}
```

Retorno fallido en caso de que el Usuario ingresa mal la contraseña

```
@Test
@DisplayName("login - debe retornar Optional vacío si la contraseña es incorrecta")
void testLogin_PasswordIncorrecto() {
    when(usuarioRepository.findByUsername(username: "admin@donaton.cl"))
        .thenReturn(Optional.of(usuarioAdmin));

    Optional<LoginResponse> resultado =
        authService.login(new LoginRequest(username: "admin@donaton.cl", password: "PasswordMalo!"));

    assertTrue(resultado.isEmpty());
    verify(jwtService, never()).generarToken(any(), any(), any(), any());
    verify(usuarioRepository, never()).save(any());
}
```


Registrarse:

Registro Exitoso de Usuario

```
@Test
@DisplayName("registrar - debe crear nuevo usuario y retornar Optional con RegisterResponse")
void testRegistrar_Exitoso() {
    Usuario guardado = Usuario.builder()
        .id(id: 3L)
        .username(username: "nuevo@donaton.cl")
        .email(email: "nuevo@donaton.cl")
        .rol(rol: "ROLE_USER")
        .nombreCompleto(nombreCompleto: "Nuevo Usuario")
        .rut(rut: "98765432-1")
        .activo(activo: true)
        .build();

    when(usuarioRepository.existsByUsername(username: "nuevo@donaton.cl")).thenReturn(value: false);
    when(usuarioRepository.existsByEmail(email: "nuevo@donaton.cl")).thenReturn(value: false);
    when(usuarioRepository.save(any(type: Usuario.class))).thenReturn(guardado);

    Optional<RegisterResponse> resultado = authService.registrar(registerRequest);

    assertTrue(resultado.isPresent());
    assertEquals(expected: "nuevo@donaton.cl", resultado.get().getUsername());
    assertEquals(expected: "ROLE_USER", resultado.get().getRol());
    assertEquals(expected: "Nuevo Usuario", resultado.get().getNombreCompleto());
    assertEquals(expected: "Usuario registrado exitosamente", resultado.get().getMensaje());
    verify(usuarioRepository, times(wantedNumberOfInvocations: 1)).save(any(type: Usuario.class));
}
```

Retorno fallido si el nombre de usuario ya existe

```
@Test
@DisplayName("registrar - debe retornar Optional vacío si el username ya existe")
void testRegistrar_UsernameDuplicado() {
    when(usuarioRepository.existsByUsername(username: "nuevo@donaton.cl")).thenReturn(value: true);

    Optional<RegisterResponse> resultado = authService.registrar(registerRequest);

    assertTrue(resultado.isEmpty());
    verify(usuarioRepository, never()).save(any());
}
```

Retorno fallido si el mail ya existe

```
@Test
@DisplayName("registrar - debe retornar Optional vacío si el email ya existe")
void testRegistrar_EmailDuplicado() {
    when(usuarioRepository.existsByUsername(username: "nuevo@donaton.cl")).thenReturn(value: false);
    when(usuarioRepository.existsByEmail(email: "nuevo@donaton.cl")).thenReturn(value: true);

    Optional<RegisterResponse> resultado = authService.registrar(registerRequest);

    assertTrue(resultado.isEmpty());
    verify(usuarioRepository, never()).save(any());
}
```

Retorno fallido si no se asigna automáticamente el ROL de usuario

```
@Test
@DisplayName("registrar - debe asignar ROLE_USER por defecto si el rol es null")
void testRegistrar_RolNullAsignaRoleUser() {
    RegisterRequest sinRol = new RegisterRequest(
        username: "sinrol@donaton.cl", password: "Pass123!",
        nombreCompleto: "Sin Rol", email: "sinrol@donaton.cl",
        rol: null, rut: null
    );

    Usuario guardado = Usuario.builder()
        .id(id: 4L)
        .username(username: "sinrol@donaton.cl")
        .email(email: "sinrol@donaton.cl")
        .rol(rol: "ROLE_USER")
        .nombreCompleto(nombreCompleto: "Sin Rol")
        .activo(activo: true)
        .build();

    when(usuarioRepository.existsByUsername(username: "sinrol@donaton.cl")).thenReturn(value: false);
    when(usuarioRepository.existsByEmail(email: "sinrol@donaton.cl")).thenReturn(value: false);
    when(usuarioRepository.save(any(type: Usuario.class))).thenReturn(guardado);

    Optional<RegisterResponse> resultado = authService.registrar(sinRol);

    assertTrue(resultado.isPresent());
    assertEquals(expected: "ROLE_USER", resultado.get().getRol());
}
```

Retorno fallido si no se ingresa el Rol automáticamente (en blanco)

```
@Test
@DisplayName("registrar - debe asignar ROLE_USER por defecto si el rol es blank")
void testRegistrar_RolBlankAsignaRoleUser() {
    RegisterRequest rolBlanco = new RegisterRequest(
        username: "blankrol@donaton.cl", password: "Pass123!",
        nombreCompleto: "Blank Rol", email: "blankrol@donaton.cl",
        rol: " ", rut: null // rol = solo espacios
    );

    Usuario guardado = Usuario.builder()
        .id(id: 5L)
        .username(username: "blankrol@donaton.cl")
        .email(email: "blankrol@donaton.cl")
        .rol(rol: "ROLE_USER")
        .nombreCompleto(nombreCompleto: "Blank Rol")
        .activo(activo: true)
        .build();

    when(usuarioRepository.existsByUsername(username: "blankrol@donaton.cl")).thenReturn(value: false);
    when(usuarioRepository.existsByEmail(email: "blankrol@donaton.cl")).thenReturn(value: false);
    when(usuarioRepository.save(any(type: Usuario.class))).thenReturn(guardado);

    Optional<RegisterResponse> resultado = authService.registrar(rolBlanco);

    assertTrue(resultado.isPresent());
    assertEquals(expected: "ROLE_USER", resultado.get().getRol());
}
```

Creación de usuario exitosa en la base de datos

```
@Test
@DisplayName("crearSiNoExiste - debe guardar el usuario si no existe en BD")
void testCrearSiNoExiste_UsuarioNuevo() {
    when(usuarioRepository.existsByUsername(username: "default@donaton.cl")).thenReturn(value: false);
    when(usuarioRepository.save(any(type: Usuario.class))).thenReturn(usuarioAdmin);

    assertDoesNotThrow(() -> authService.crearSiNoExiste(
        username: "default@donaton.cl", rawPassword: "password123",
        rol: "ADMIN", nombreCompleto: "Default Admin",
        email: "default@donaton.cl", rut: "11111111-1"
    ));

    verify(usuarioRepository, times(wantedNumberOfInvocations: 1)).save(any(type: Usuario.class));
}
```

Retorno de creación de usuario fallido si el usuario ya existe

```
@Test
@DisplayName("crearSiNoExiste - debe omitir la creación si el usuario ya existe")
void testCrearSiNoExiste_UsuarioYaExiste() {
    when(usuarioRepository.existsByUsername(username: "admin@donaton.cl")).thenReturn(value: true);

    assertDoesNotThrow(() -> authService.crearSiNoExiste(
        username: "admin@donaton.cl", rawPassword: "Admin1234!",
        rol: "ADMIN", nombreCompleto: "Administrador Sistema",
        email: "admin@donaton.cl", rut: "12345678-9"
    ));

    verify(usuarioRepository, never()).save(any());
}
```

Registro de Donante ya existe

```
// -----Test:RegistrarDonante-----
@Test
@DisplayName("Registrar donante - debe retornar donante existente si ya está registrado por RUT")
void testRegistrarDonante_DonanteYaExiste() {
    //El rut nomralizado de "98765432-1" es "98765432-1"
    when(donanteRepository.findByRut(rut: "98765432-1")).thenReturn(Optional.of(particular));

    DonanteResponseDTO resultado = donacionService.registrarDonante(donanteParticularDTO);
    You, 2 weeks ago • test Donacion Service
    assertNotNull(resultado);
    //Cuando ya Existe, retorna sin guardar
    verify(donanteRepository, never()).save(any());
}
```

Creación de donante Particular exitoso

```
@Test
@DisplayName("Registrar donante - debe crear donante PARTICULAR si no existe")
void testRegistrarDonante_NuevoParticular(){
    Particular nuevo = new Particular();
    nuevo.setId(id: 3L);
    nuevo.setRut(rut: "98765432-1");
    nuevo.setNombres(nombres: "María");
    nuevo.setApellidos(apellidos: "González");
    nuevo.setEmail(email: "maria.gonzalez@email.com");

    when(donanteRepository.findByRut(rut: "98765432-1")).thenReturn(Optional.empty());
    when(donanteRepository.save(any(type: Donante.class))).thenReturn(nuevo);

    DonanteResponseDTO resultado = donacionService.registrarDonante(donanteParticularDTO);

    assertNotNull(resultado);
    assertEquals(expected: "98765432-1", resultado.getRut());
    assertEquals(expected: "PARTICULAR", resultado.getTipoDonante());
    verify(donanteRepository, times(wantedNumberOfInvocations: 1)).save(any(type: Donante.class));
}
```

Registra un Donante Empresa exitoso

```
@Test
@DisplayName("Registrar donante - debe crear donante EMPRESA si no existe")
void testRegistrarDonante_NuevaEmpresa(){
    Empresa nueva = new Empresa();
    nueva.setId(id: 4L);
    nueva.setRut(rut: "65432100-5");
    nueva.setRazonSocial(razonSocial: "TechCorp SpA");
    nueva.setEmail(email: "donaciones@techcorp.cl");

    when(donanteRepository.findByRut(rut: "65432100-5")).thenReturn(Optional.empty());
    when(donanteRepository.save(any(type: Donante.class))).thenReturn(nueva);

    DonanteResponseDTO resultado = donacionService.registrarDonante(donanteEmpresaDTO);

    assertNotNull(resultado);
    assertEquals(expected: "65432100-5", resultado.getRut());
    verify(donanteRepository, times(wantedNumberOfInvocations: 1)).save(any(type: Donante.class));
}
```

Si el tipo de donante es inválido, manda una excepción

```
@Test
@DisplayName("Registrar donante - debe lanzar excepción si el tipo de donante es inválido")
void testRegistrarDonante_TipoInvalido(){
    DonanteRequestDTO dtoInvalido = new DonanteRequestDTO();
    dtoInvalido.setTipoDonante(tipoDonante: "ANONIMO");
    dtoInvalido.setRut(rut: "11111111-1");
    dtoInvalido.setEmail(email: "test@test.com");

    when (donanteRepository.findByRut(rut: "11111111-1")).thenReturn(Optional.empty());

    assertThrows(expectedType: IllegalArgumentException.class, () -> donacionService.registrarDonante(dtoInvalido));

    verify(donanteRepository, never()).save(any());
}
```

Al momento de registrar PARTICULAR, lanza excepción si falta nombre

```
@Test
@DisplayName("registrarDonante PARTICULAR - debe lanzar excepción si faltan nombres")
void testRegistrarDonante_ParticularSinNombres(){
    DonanteRequestDTO dto = new DonanteRequestDTO();
    dto.setTipoDonante(tipoDonante: "PARTICULAR");
    dto.setRut(rut: "11111111-1");
    dto.setEmail(email: "test@test.com");
    //Nombres y apellido en null

    when(donanteRepository.findByRut(rut: "11111111-1")).thenReturn(Optional.empty());

    assertThrows(expectedType: IllegalArgumentException.class, () -> donacionService.registrarDonante(dto));
}
```

Registro de Empresa lanza excepción si falta razón social

```

@Test
@DisplayName("registrarDonante EMPRESA - debe lanzar excepción si faltan razón social")
void testRegistrarDonante_EmpresaSinRazonSocial(){
    DonanteRequestDTO dto = new DonanteRequestDTO();
    dto.setTipoDonante(tipoDonante: "EMPRESA");
    dto.setRut(rut: "11111111-1");
    dto.setEmail(email: "test@test.com");
    //razon social y nombreContacto en null

    when(donanteRepository.findByRut(rut: "11111111-1")).thenReturn(Optional.empty());

    assertThrows(expectedType: IllegalArgumentException.class, () -> donacionService.registrarDonante(dto));
}

```

Retorna Lista de todos los donantes (Particulares y Empresa)

```

@Test
@DisplayName("Listar donantes - debe retornar lista de todos los donantes")
void testListarDonantes_Exitoso(){
    when(donanteRepository.findAll()).thenReturn(Arrays.asList(particular, empresa));

    List<DonanteResponseDTO> resultado = donacionService.listarDonantes();

    assertNotNull(resultado);
    assertEquals(expected: 2, resultado.size());
    verify(donanteRepository, times(wantedNumberOfInvocations: 1)).findAll();
}

```

Retorna una lista vacía si no hay donantes

```

@Test
@DisplayName("ListarDonantes - debe retornar lista vacía si no hay donantes")
void testListarDonantes_ListaVacía(){
    when(donanteRepository.findAll()).thenReturn(List.of());

    List<DonanteResponseDTO> resultado = donacionService.listarDonantes();

    assertNotNull(resultado);
    assertTrue(resultado.isEmpty());
}

```

Filtro de donante por rut

```

@Test
@DisplayName("obtenerDonantePorRut - debe retornar donante existente")
void testObtenerDonantePorRut_Exitoso(){
    when(donanteRepository.findByRut(rut: "12345678-9")).thenReturn(Optional.of(particular));

    DonanteResponseDTO resultado = donacionService.obtenerDonantePorRut(rut: "12345678-9");

    assertNotNull(resultado);
    assertEquals(expected: "12345678-9", resultado.getRut());
}

```

Retorno fallido la búsqueda si no existe el rut

```

@Test
@DisplayName("obtenerDonantePorRut - debe lanzar excepción si el rut no existe")
void testObtenerDonantePorRut_NoEncontrado(){
    when(donanteRepository.findByRut(rut: "99999999-9")).thenReturn(Optional.empty());

    assertThrows(expectedType: IllegalArgumentException.class, () -> donacionService.obtenerDonantePorRut(rut: "99999999-9"));
}

```

Actualización de campos del donante


```

@Test
@DisplayName("actualizarDonante - debe actualizar campos del donante exitosamente")
void testActualizarDonante_Exitoso(){
    DonanteRequestDTO actualizacion = new DonanteRequestDTO();
    actualizacion.setTipoDonante(tipoDonante: "PARTICULAR");
    actualizacion.setRut(rut: "12345678-9");
    actualizacion.setEmail(email: "nuevo@email.com");
    actualizacion.setTelefono(telefono: "999999999");
    actualizacion.setNombres(nombres: "Juan Carlos");
    actualizacion.setApellidos(apellidos: "Pérez López");

    Particular actualizado = new Particular();
    actualizado.setId(id: 1L);
    actualizado.setRut(rut: "12345678-9");
    actualizado.setEmail(email: "nuevo@email.com");
    actualizado.setNombres(nombres: "Juan Carlos");

    when(donanteRepository.findById(id: 1L)).thenReturn(Optional.of(particular));
    when(donanteRepository.save(any(type: Donante.class))).thenReturn(actualizado);

    DonanteResponseDTO resultado = donacionService.actualizarDonante(id: 1L, actualizacion);

    assertNotNull(resultado);
    verify(donanteRepository, times(wantedNumberOfInvocations: 1)).save(any(type: Donante.class));
}

```

Actualización de excepción si el donante no existe

```

@Test
@DisplayName("actualizarDonante - debe lanzar excepción si el donante no existe")
void testActualizarDonante_NoEncontrado(){
    when(donanteRepository.findById(id: 99L)).thenReturn(Optional.empty());

    assertThrows(expectedType: IllegalArgumentException.class, () -> donacionService.actualizarDonante(id: 99L, donanteParticularDTO));

    verify(donanteRepository, never()).save(any());
}

```

Eliminar exitosamente un donante

```

@Test
@DisplayName("eliminarDonante - debe eliminar donante existente exitosamente")
void testEliminarDonante_exitoso(){
    when(donanteRepository.existsById(id: 1L)).thenReturn(value: true);
    doNothing().when(donanteRepository).deleteById(id: 1L);

    assertDoesNotThrow(() -> donacionService.eliminarDonante(id: 1L));

    verify(donanteRepository, times(wantedNumberOfInvocations: 1)).deleteById(id: 1L);
}

```

Retorno fallido si no se puede eliminar donante

```

@Test
@DisplayName("eliminarDonante - debe lanzar excepción si el donante no existe")
void testEliminarDonante_NoEncontrado(){
    when(donanteRepository.existsById(id: 99L)).thenReturn(value: false);

    assertThrows(expectedType: IllegalArgumentException.class, () -> donacionService.eliminarDonante(id: 99L));

    verify(donanteRepository, never()).deleteById(any());
}

```

Elimina recurso donado existente

```

@Test
@DisplayName("eliminarRecurso - debe eliminar recurso donado existente")
void testEliminarRecurso_Exitoso(){
    when (recursoDonadoRepository.existsById(id: 1L)).thenReturn(value: true);
    doNothing().when(recursoDonadoRepository).deleteById(id: 1L);

    assertDoesNotThrow(() -> donacionService.eliminarRecurso(id: 1L));

    verify(recursoDonadoRepository, times(wantedNumberOfInvocations: 1)).deleteById(id: 1L);
}

```

Retorna excepción si el recurso no existe

```
@Test
@DisplayName("eliminarRecurso - debe lanzar excepción si el recurso no existe")
void testEliminarRecurso_NoEncontrado(){
    when(recursoDonadoRepository.existsById(id: 99L)).thenReturn(value: false);

    assertThrows(expectedType: IllegalArgumentException.class, () -> donacionService.eliminarRecurso(id: 99L));

    verify(recursoDonadoRepository, never()).deleteById(any());
}
```

Registra donación con Donante existente

```
//-----Test: procesarDonacion-----
@Test
@DisplayName("procesarDonacion - debe registrar donación con donante existente")
void testProcesarDonacion_DonanteExistente(){
    when(donanteRepository.existsByRut(rut: "12345678-9")).thenReturn(value: true);
    when(donanteRepository.findByRut(rut: "12345678-9")).thenReturn(Optional.of(particular));
    when(recursoDonadoRepository.save(any(type: RecursoDonado.class))).thenAnswer(invoc
        -> {
            RecursoDonado r = invoc.getArgument(index: 0);
            r.setId(id: 10L);
            return r;
        });

    DonacionResponseDTO resultado = donacionService.procesarDonacion(donacionParticularDTO);

    assertNotNull(resultado);
    assertFalse(resultado.isDonanteNuevo());
    assertEquals(TipoRecurso.ROPA, resultado.getTipoRecurso());
    assertEquals(new BigDecimal("20.00"), resultado.getCantidad());
    verify(recursoDonadoRepository, times(wantedNumberOfInvocations: 1)).save(any(type: RecursoDonado.class));
}
```

Crea nuevo y registro de donación

```
@Test
@DisplayName("procesarDonacion - debe crear donante nuevo y registrar donación ")
void testProcesarDonacion_DonanteNuevo(){
    Particular nuevoParticular = new Particular();
    nuevoParticular.setId(id: 5L);
    nuevoParticular.setRut(rut: "98765432-1");
    nuevoParticular.setNombres(nombres: "Ana");
    nuevoParticular.setApellidos(apellidos: "López");

    DonacionRequestDTO dto = new DonacionRequestDTO();
    dto.setTipoDonante(tipoDonante: "PARTICULAR");
    dto.setRut(rut: "98765432-1");
    dto.setEmail(email: "ana@test.com");
    dto.setNombres(nombres: "Ana");
    dto.setApellidos(apellidos: "López");
    dto.setTipoRecurso(TipoRecurso.ALIMENTOS);
    dto.setCantidad(new BigDecimal("30.00"));
    dto.setUnidadMedida(unidadMedida: "kg");

    when(donanteRepository.existsByRut(rut: "98765432-1")).thenReturn(value: false);
    when(donanteRepository.findByRut(rut: "98765432-1")).thenReturn(Optional.empty());
    when(donanteRepository.save(any(type: Donante.class))).thenReturn(nuevoParticular);
    when(recursoDonadoRepository.save(any(type: RecursoDonado.class))).thenAnswer(invoc
        -> {
            RecursoDonado r = invoc.getArgument(index: 0);
            r.setId(id: 11L);
            return r;
        });

    DonacionResponseDTO resultado = donacionService.procesarDonacion(dto);

    assertNotNull(resultado);
    assertTrue(resultado.isDonanteNuevo());
    assertEquals(TipoRecurso.ALIMENTOS, resultado.getTipoRecurso());
    //Se guarda el donante + el recurso
    verify(donanteRepository, times(wantedNumberOfInvocations: 1)).save(any(type: Donante.class));
    verify(recursoDonadoRepository, times(wantedNumberOfInvocations: 1)).save(any(type: RecursoDonado.class));
}
```

Retorno lanza excepción si no reconoce donante

```
@Test
@DisplayName("procesarDonacion - debe lanzar excepción si tipo donante no reconocido")
void testProcesarDonacion_TipoInvalido(){
    DonacionRequestDTO dto = new DonacionRequestDTO();
    dto.setTipoDonante(tipoDonante: "DESCONOCIDO");
    dto.setRut(rut: "11111111-1");
    dto.setEmail(email: "x@x.com");
    dto.setTipoRecurso(TipoRecurso.ROPA);
    dto.setCantidad(BigDecimal.ONE);
    dto.setUnidadMedida(unidadMedida: "kg");

    when(donanteRepository.existsByRut(rut: "11111111-1")).thenReturn(value: false);
    when(donanteRepository.findByRut(rut: "11111111-1")).thenReturn(Optional.empty());

    assertThrows(expectedType: IllegalArgumentException.class, () -> donacionService.procesarDonacion(dto));
}
```

Retorno la lista de los recursos donados

```
@Test
@DisplayName("obtenerHistorialPorRut - debe retornar lista de recursos del donante")
void testObtenerHistorialPorRut_Exitoso(){
    when (recursoDonadoRepository.findHistorialByRut(rut: "12345678-9")).thenReturn(List.of(recursoDonado));

    List<RecursoDonado> resultado = donacionService.obtenerHistorialPorRut(rut: "12345678-9");

    assertNotNull(resultado);
    assertEquals(expected: 1, resultado.size());
    assertEquals(TipoRecurso.ALIMENTOS, resultado.get(0).getTipoRecurso());
}
```

Retorna lista vacía si no hay historial

```
@Test
@DisplayName("obtenerHistorialPorRut - debe retornar lista vacía si no hay historial")
void testObtenerHistorialPorRut_Vacio(){
    when(recursoDonadoRepository.findHistorialByRut(rut: "99999999-9")).thenReturn(List.of());

    List<RecursoDonado> resultado = donacionService.obtenerHistorialPorRut(rut: "99999999-9");

    assertNotNull(resultado);
    assertTrue(resultado.isEmpty());
}
```

Retorna página de recursos

```
@Test
@DisplayName("obtenerHistorialPorRutPaginado - debe retornar página de recursos")
void testObtenerHistorialPorRutPaginado_Exitoso(){
    Pageable pageable = PageRequest.of(pageNumber: 0, pageSize: 10);
    Page<RecursoDonado> pagina = new PageImpl<>(List.of(recursoDonado));

    when(recursoDonadoRepository.findHistorialByRutPaginado(rut: "12345678-9", pageable)).thenReturn(pagina);
    Page<RecursoDonado> resultado = donacionService.obtenerHistorialPorRutPaginado(rut: "12345678-9", pageable);

    assertNotNull(resultado);
    assertEquals(expected: 1, resultado.getTotalElements());
}
```

Retorna todas las donaciones paginadas

```
@Test
@DisplayName("obtenerHistorialGlobal - debe retornar todas las donaciones paginadas")
void testObtenerHistorialGlobal_Exitoso(){
    Pageable pageable = PageRequest.of(pageNumber: 0, pageSize: 10);
    RecursoDonado recurso2 = new RecursoDonado();
    recurso2.setId(id: 2L);
    recurso2.setTipoRecurso(TipoRecurso.DINERO);
    recurso2.setCantidad(new BigDecimal("5000.00"));
    recurso2.setUnidadMedida(unidadMedida: "CLP");
    recurso2.setDonante(empresa);

    Page<RecursoDonado> pagina = new PageImpl<>(Arrays.asList(recursoDonado, recurso2));

    when(recursoDonadoRepository.findAllConDonante(pageable)).thenReturn(pagina);

    Page<RecursoDonado> resultado = donacionService.obtenerHistorialGlobal(pageable);
    assertNotNull(resultado);
    assertEquals(expected: 2, resultado.getTotalElements());
    verify(recursoDonadoRepository, times(wantedNumberOfInvocations: 1)).findAllConDonante(pageable);
}
```

Logística Test:

Retorno exitoso de creación de Centro Acopio

```
@Test
@DisplayName("crearCentroAcopio - debe persistir y retornar el centro creado")
void testCrearCentroAcopio_Exitoso() {
    CentroAcopio guardado = new CentroAcopio();
    guardado.setId(id: 3L);
    guardado.setNombre(centroRequest.getNombre());
    guardado.setDireccion(centroRequest.getDireccion());
    guardado.setRegion(centroRequest.getRegion());
    guardado.setComuna(centroRequest.getComuna());
    guardado.setEstado(EstadoCentro.ACTIVO);
    guardado.setCapacidadMaximaKg(centroRequest.getCapacidadMaximaKg());
    guardado.setInventario(new ArrayList<>());

    when(centroRepo.save(any(type: CentroAcopio.class))).thenReturn(guardado);

    CentroAcopioResponse resultado = logisticaService.crearCentroAcopio(centroRequest);

    assertNotNull(resultado);
    assertEquals(expected: 3L, resultado.getId());
    assertEquals(expected: "Centro Las Condes", resultado.getNombre());
    assertEquals(expected: "ACTIVO", resultado.getEstado());
    verify(centroRepo, times(wantedNumberOfInvocations: 1)).save(any(type: CentroAcopio.class));
}
```

Obtenemos el Centro de acopio y su inventario

```
@Test
@DisplayName("obtenerCentroPorId - debe retornar el centro si existe (usa findByIdWithInventario)")
void testObtenerCentroPorId_Exitoso() {
    // El service usa findByIdWithInventario (no findById) para traer el inventario
    when(centroRepo.findByIdWithInventario(id: 1L)).thenReturn(Optional.of(centroActivo));

    CentroAcopioResponse resultado = logisticaService.obtenerCentroPorId(id: 1L);

    assertNotNull(resultado);
    assertEquals(expected: 1L, resultado.getId());
    assertEquals(expected: "Centro Santiago Centro", resultado.getNombre());
    assertEquals(expected: "ACTIVO", resultado.getEstado());
    // El inventario del centro activo tiene 2 items
    assertEquals(expected: 2, resultado.getInventario().size());
    verify(centroRepo, times(wantedNumberOfInvocations: 1)).findByIdWithInventario(id: 1L);
}
```

Lanza IllegalArgumentException si no existe

```
@Test
@DisplayName("obtenerCentroPorId - debe lanzar IllegalArgumentException si no existe")
void testObtenerCentroPorId_NoEncontrado() {
    when(centroRepo.findByIdWithInventario(id: 99L)).thenReturn(Optional.empty());

    assertThrows(expectedType: IllegalArgumentException.class,
        () -> logisticaService.obtenerCentroPorId(id: 99L));
}
```

Retorna listas de Activos

```
@Test
@DisplayName("listarCentrosActivos - debe consultar por estado ACTIVO y retornar la lista")
void testListarCentrosActivos_Exitoso() {
    when(centroRepo.findByEstadoOrderByNombre(EstadoCentro.ACTIVO))
        .thenReturn(List.of(centroActivo));

    List<CentroAcopioResponse> resultado = logisticaService.listarCentrosActivos();

    assertNotNull(resultado);
    assertEquals(expected: 1, resultado.size());
    assertEquals(expected: "ACTIVO", resultado.get(0).getEstado());
    verify(centroRepo, times(wantedNumberOfInvocations: 1)).findByEstadoOrderByNombre(EstadoCentro.ACTIVO);
}
```

Retorna lista vacía si no hay centros activos

```
@Test
@DisplayName("listarCentrosActivos - debe retornar lista vacía si no hay centros activos")
void testListarCentrosActivos_Vacio() {
    when(centroRepo.findByEstadoOrderByNombre(EstadoCentro.ACTIVO))
        .thenReturn(List.of());

    List<CentroAcopioResponse> resultado = logisticaService.listarCentrosActivos();

    assertNotNull(resultado);
    assertTrue(resultado.isEmpty());
}
```


Listar Centros por región

```
@Test
@DisplayName("listarCentrosPorRegion - debe consultar por región ignorando mayúsculas")
void testListarCentrosPorRegion_Exitoso() {
    when(centroRepo.findByRegionIgnoreCaseOrderByNombre(region: "Metropolitana"))
        .thenReturn(Arrays.asList(centroActivo, centroInactivo));

    List<CentroAcopioResponse> resultado =
        logisticaService.listarCentrosPorRegion(region: "Metropolitana");

    assertNotNull(resultado);
    assertEquals(expected: 2, resultado.size());
    verify(centroRepo, times(wantedNumberOfInvocations: 1))
        .findByRegionIgnoreCaseOrderByNombre(region: "Metropolitana");
}
```

Listar Centros por región ignorando mayúsculas

```
@Test
@DisplayName("listarCentrosPorRegion - debe consultar por región ignorando mayúsculas")
void testListarCentrosPorRegion_Exitoso() {
    when(centroRepo.findByRegionIgnoreCaseOrderByNombre(region: "Metropolitana"))
        .thenReturn(Arrays.asList(centroActivo, centroInactivo));

    List<CentroAcopioResponse> resultado =
        logisticaService.listarCentrosPorRegion(region: "Metropolitana");

    assertNotNull(resultado);
    assertEquals(expected: 2, resultado.size());
    verify(centroRepo, times(wantedNumberOfInvocations: 1))
        .findByRegionIgnoreCaseOrderByNombre(region: "Metropolitana");
}
```

Retorna todos los centros sin filtro

```
@Test
@DisplayName("listarTodosLosCentros - debe retornar todos los centros sin filtro")
void testListarTodosLosCentros_Exitoso() {
    when(centroRepo.findAll()).thenReturn(Arrays.asList(centroActivo, centroInactivo));

    List<CentroAcopioResponse> resultado = logisticaService.listarTodosLosCentros();

    assertNotNull(resultado);
    assertEquals(expected: 2, resultado.size());
    verify(centroRepo, times(wantedNumberOfInvocations: 1)).findAll();
}
```

Actualiza centros de acopio

```

@Test
@DisplayName("actualizarCentro - debe actualizar campos y retornar la respuesta")
void testActualizarCentro_Exitoso() {
    CentroAcopioRequest actualizacion = new CentroAcopioRequest();
    actualizacion.setNombre(nombre: "Centro Santiago Centro v2");
    actualizacion.setDireccion(direccion: "Nueva dirección 999");
    actualizacion.setRegion(region: "Metropolitana");
    actualizacion.setComuna(comuna: "Santiago");
    actualizacion.setCapacidadMaximaKg(capacidadMaximaKg: 8000);
    actualizacion.setEmailEncargado(emailEncargado: "nuevo@donaton.cl");
    actualizacion.setNombreEncargado(nombreEncargado: "Pedro Encargado");

    CentroAcopio actualizado = new CentroAcopio();
    actualizado.setId(id: 1L);
    actualizado.setNombre(nombre: "Centro Santiago Centro v2");
    actualizado.setDireccion(direccion: "Nueva dirección 999");
    actualizado.setRegion(region: "Metropolitana");
    actualizado.setComuna(comuna: "Santiago");
    actualizado.setEstado(EstadoCentro.ACTIVO);
    actualizado.setCapacidadMaximaKg(capacidadMaximaKg: 8000);
    actualizado.setInventario(new ArrayList<>());

    when(centroRepo.findById(id: 1L)).thenReturn(Optional.of(centroActivo));
    when(centroRepo.save(any(type: CentroAcopio.class))).thenReturn(actualizado);

    CentroAcopioResponse resultado = logisticaService.actualizarCentro(id: 1L, actualizacion);

    assertNotNull(resultado);
    assertEquals(expected: "Centro Santiago Centro v2", resultado.getNombre());
    assertEquals(expected: 8000, resultado.getCapacidadMaximaKg());
    verify(centroRepo, times(wantedNumberOfInvocations: 1)).save(any(type: CentroAcopio.class));
}

```

Retorna `IllegalArgumentException` si el centro no existe

```

@Test
@DisplayName("actualizarCentro - debe lanzar IllegalArgumentException si el centro no existe")
void testActualizarCentro_NoEncontrado() {
    when(centroRepo.findById(id: 99L)).thenReturn(Optional.empty());

    assertThrows(expectedType: IllegalArgumentException.class,
        () -> logisticaService.actualizarCentro(id: 99L, centroRequest));

    verify(centroRepo, never()).save(any());
}

```

Boolean de Activo o Inactivo

```

@Test
@DisplayName("toggleEstadoCentro - debe cambiar de ACTIVO a INACTIVO")
void testToggleEstadoCentro_ActivoAInactivo() {
    CentroAcopio resultado = new CentroAcopio();
    resultado.setId(id: 1L);
    resultado.setNombre(nombre: "Centro Santiago Centro");
    resultado.setDireccion(direccion: "Av. O'Higgins 1234");
    resultado.setRegion(region: "Metropolitana");
    resultado.setComuna(comuna: "Santiago");
    resultado.setEstado(EstadoCentro.INACTIVO);
    resultado.setInventario(new ArrayList<>());

    when(centroRepo.findById(id: 1L)).thenReturn(Optional.of(centroActivo));
    when(centroRepo.save(any(type: CentroAcopio.class))).thenReturn(resultado);

    CentroAcopioResponse response = logisticaService.toggleEstadoCentro(id: 1L);

    // El centro estaba ACTIVO → queda INACTIVO
    assertEquals(expected: "INACTIVO", response.getEstado());
    verify(centroRepo, times(wantedNumberOfInvocations: 1)).save(any(type: CentroAcopio.class));
}

```

Boolean de Inactivo a Activo

```

@Test
@DisplayName("toggleEstadoCentro - debe cambiar de INACTIVO a ACTIVO")
void testToggleEstadoCentro_InactivoAActivo() {
    CentroAcopio resultadoActivo = new CentroAcopio();
    resultadoActivo.setId(id: 2L);
    resultadoActivo.setNombre(nombre: "Centro Providencia");
    resultadoActivo.setDireccion(direccion: "Av. Providencia 5678");
    resultadoActivo.setRegion(region: "Metropolitana");
    resultadoActivo.setComuna(comuna: "Providencia");
    resultadoActivo.setEstado(EstadoCentro.ACTIVO);
    resultadoActivo.setInventario(new ArrayList<>());

    when(centroRepo.findById(id: 2L)).thenReturn(Optional.of(centroInactivo));
    when(centroRepo.save(any(type: CentroAcopio.class))).thenReturn(resultadoActivo);

    CentroAcopioResponse response = logisticaService.toggleEstadoCentro(id: 2L);

    assertEquals(expected: "ACTIVO", response.getEstado());
}

```

Eliminar centro de Acopio exitoso

```

@Test
@DisplayName("eliminarCentro - debe eliminar si el centro existe")
void testEliminarCentro_Exitoso() {
    when(centroRepo.existsById(id: 1L)).thenReturn(value: true);
    doNothing().when(centroRepo).deleteById(id: 1L);

    assertDoesNotThrow(() -> logisticaService.eliminarCentro(id: 1L));

    verify(centroRepo, times(wantedNumberOfInvocations: 1)).deleteById(id: 1L);
}

```

Suma stock a item existent exits

```

@Test
@DisplayName("recibirDonacion (integración WebClient) - debe sumar stock a item existente")
void testRecibirDonacion_ItemExistente() {
    // Simula que el ApiGateway ya llamó a MS Donacion y ahora notifica a Logistica
    BigDecimal cantidadRecibida = new BigDecimal("50.00");
    BigDecimal stockPrevio = itemAlimentos.getCantidadDisponible(); // 200.00

    when(centroRepo.findById(id: 1L)).thenReturn(Optional.of(centroActivo));
    when(inventarioRepo.findByCentroAcopioIdAndTipoRecurso(centroAcopioId: 1L, TipoRecurso.ALIMENTOS))
        .thenReturn(Optional.of(itemAlimentos));
    when(inventarioRepo.save(any(type: ItemInventario.class))).thenReturn(itemAlimentos);

    assertDoesNotThrow(() ->
        logisticaService.recibirDonacion(centroId: 1L, tipoRecursoStr: "ALIMENTOS", cantidadRecibida, unidadMedida: "kg"));

    // agregarStock modifica el objeto en memoria: 200 + 50 = 250
    assertEquals(stockPrevio.add(cantidadRecibida), itemAlimentos.getCantidadDisponible());
    verify(inventarioRepo, times(wantedNumberOfInvocations: 1)).save(itemAlimentos);
}

```

Crea item nuevo si no existe en inventario

```

@Test
@DisplayName("recibirDonacion (integración WebClient) - debe crear item nuevo si no existe en inventario")
void testRecibirDonacion_ItemNuevo() {
    // Primera donación de INSUMOS_MEDICOS a este centro + no existe item previo
    when(centroRepo.findById(id: 1L)).thenReturn(Optional.of(centroActivo));
    when(inventarioRepo.findByCentroAcopioIdAndTipoRecurso(centroAcopioId: 1L, TipoRecurso.INSUMOS_MEDICOS))
        .thenReturn(Optional.empty()); // aún no existe
    when(inventarioRepo.save(any(type: ItemInventario.class))).thenAnswer(invoc -> {
        ItemInventario nuevo = invoc.getArgument(index: 0);
        nuevo.setId(id: 10L);
        return nuevo;
    });

    assertDoesNotThrow(() ->
        logisticaService.recibirDonacion(centroId: 1L, tipoRecursoStr: "INSUMOS_MEDICOS",
            new BigDecimal("20.00"), unidadMedida: "unidades"));

    // Se guardó un item nuevo con el stock inicial de 20
    verify(inventarioRepo, times(wantedNumberOfInvocations: 1)).save(any(type: ItemInventario.class));
}

```

Si el centro no existe al recibir donación, lanza IllegalArgumentException

```

@Test
@DisplayName("recibirDonacion - debe lanzar IllegalArgumentException si el centro no existe")
void testRecibirDonacion_CentroNoEncontrado() {
    when(centroRepo.findById(id: 99L)).thenReturn(Optional.empty());

    assertThrows(expectedType: IllegalArgumentException.class,
        () -> logisticaService.recibirDonacion(centroId: 99L, tipoRecursoStr: "ALIMENTOS",
            new BigDecimal("10.00"), unidadMedida: "kg"));

    verify(inventarioRepo, never()).save(any());
}

```

Planificación de envío descuento stock y retorna respuesta

```

@Test
@DisplayName("planificarEnvio - debe descontar stock, crear envío y retornar respuesta")
void testPlanificarEnvio_Exitoso() {
    // El centro trae inventario con 200 kg de ALIMENTOS; pedimos 30 kg
    when(centroRepo.findByIdWithInventario(id: 1L)).thenReturn(Optional.of(centroActivo));
    when(inventarioRepo.findByCentroAcopioIdAndTipoRecurso(centroAcopioId: 1L, TipoRecurso.ALIMENTOS))
        .thenReturn(Optional.of(itemAlimentos));
    when(inventarioRepo.save(any(type: ItemInventario.class))).thenReturn(itemAlimentos);
    when(envioRepo.save(any(type: Envio.class))).thenAnswer(invoc -> {
        Envio e = invoc.getArgument(index: 0);
        e.setId(id: 10L);
        return e;
    });

    EnvioResponse resultado = logisticaService.planificarEnvio(envioRequest);

    assertNotNull(resultado);
    assertEquals(expected: 10L, resultado.getId());
    assertEquals(expected: "PLANIFICADO", resultado.getEstado());
    assertEquals(expected: 1L, resultado.getCentroOrigenId());
    // Stock descontado: 200 - 30 = 170
    assertEquals(new BigDecimal("170.00"), itemAlimentos.getCantidadDisponible());
    verify(inventarioRepo, times(wantedNumberOfInvocations: 1)).save(itemAlimentos);
    verify(envioRepo, times(wantedNumberOfInvocations: 1)).save(any(type: Envio.class));
}

```

IllegalArgumentException si el centro no existe

```

@Test
@DisplayName("planificarEnvio - debe lanzar IllegalArgumentException si el centro no existe")
void testPlanificarEnvio_CentroNoEncontrado() {
    when(centroRepo.findByIdWithInventario(id: 1L)).thenReturn(Optional.empty());

    assertThrows(expectedType: IllegalArgumentException.class,
        () -> logisticaService.planificarEnvio(envioRequest));

    verify(envioRepo, never()).save(any());
}

```

IllegalArgumentException si no hay stock del recurso

```

@Test
@DisplayName("planificarEnvio - debe lanzar IllegalStateException si no hay stock del recurso")
void testPlanificarEnvio_SinStockEnInventario() {
    // No existe ningún ítem de ALIMENTOS en este centro
    when(centroRepo.findByIdWithInventario(id: 1L)).thenReturn(Optional.of(centroActivo));
    when(inventarioRepo.findByCentroAcopioIdAndTipoRecurso(centroAcopioId: 1L, TipoRecurso.ALIMENTOS))
        .thenReturn(Optional.empty());

    assertThrows(expectedType: IllegalStateException.class,
        () -> logisticaService.planificarEnvio(envioRequest));

    verify(envioRepo, never()).save(any());
}

```

Retorno fallido si el stock es insuficiente

```

@Test
@DisplayName("planificarEnvio - debe lanzar IllegalStateException si el stock es insuficiente")
void testPlanificarEnvio_StockInsuficiente() {
    // Solo hay 10 kg pero el envío pide 30
    itemAlimentos.setCantidadDisponible(new BigDecimal("10.00"));

    when(centroRepo.findByIdWithInventario(id: 1L)).thenReturn(Optional.of(centroActivo));
    when(inventarioRepo.findByCentroAcopioIdAndTipoRecurso(centroAcopioId: 1L, TipoRecurso.ALIMENTOS))
        .thenReturn(Optional.of(itemAlimentos));

    // descontarStock dentro de ItemInventario lanza IllegalStateException
    assertThrows(expectedType: IllegalStateException.class,
        () -> logisticaService.planificarEnvio(envioRequest));

    verify(envioRepo, never()).save(any());
}

```


Cambio de Estado en el despacho

```
@Test
@DisplayName("despacharEnvio - debe cambiar estado de PLANIFICADO a EN_TRANSITO")
void testDespacharEnvio_Exitoso() {
    // despacharEnvio usa findByIdWithDetalle internamente
    when(envioRepo.findByIdWithDetalle(id: 1L)).thenReturn(Optional.of(envioPlanificado));
    when(envioRepo.save(any(type: Envio.class))).thenAnswer(invoc -> invoc.getArgument(index: 0));

    EnvioResponse resultado = logisticaService.despacharEnvio(envioId: 1L);

    assertNotNull(resultado);
    assertEquals(expected: "EN_TRANSITO", resultado.getEstado());
    assertNotNull(resultado.getFechaDespacho());
    verify(envioRepo, times(wantedNumberOfInvocations: 1)).save(envioPlanificado);
}
```

IllegalArgumentException si el envío no está PLANIFICADO

```
@Test
@DisplayName("despacharEnvio - debe lanzar IllegalStateException si el envío no está PLANIFICADO")
void testDespacharEnvio_EstadoInvalido() {
    // envioEnTransito ya está EN_TRANSITO, no se puede despachar de nuevo
    when(envioRepo.findByIdWithDetalle(id: 2L)).thenReturn(Optional.of(envioEnTransito));

    assertThrows(expectedType: IllegalStateException.class,
        () -> logisticaService.despacharEnvio(envioId: 2L));

    verify(envioRepo, never()).save(any());
}
```

IllegalArgumentException si el envío no existe

```
@Test
@DisplayName("despacharEnvio - debe lanzar IllegalArgumentException si el envío no existe")
void testDespacharEnvio_NoEncontrado() {
    when(envioRepo.findByIdWithDetalle(id: 99L)).thenReturn(Optional.empty());

    assertThrows(expectedType: IllegalArgumentException.class,
        () -> logisticaService.despacharEnvio(envioId: 99L));
}
```

Cambio de estado exitoso

```
@Test
@DisplayName("confirmarEntrega - debe cambiar estado de EN_TRANSITO a ENTREGADO")
void testConfirmarEntrega_Exitoso() {
    ActualizarEstadoRequest request = new ActualizarEstadoRequest();
    request.setObservaciones(observaciones: "Entrega realizada sin inconvenientes");

    when(envioRepo.findByIdWithDetalle(id: 2L)).thenReturn(Optional.of(envioEnTransito));
    when(envioRepo.save(any(type: Envio.class))).thenAnswer(invoc -> invoc.getArgument(index: 0));

    EnvioResponse resultado = logisticaService.confirmarEntrega(envioId: 2L, request);

    assertNotNull(resultado);
    assertEquals(expected: "ENTREGADO", resultado.getEstado());
    assertNotNull(resultado.getFechaEntrega());
    assertEquals(expected: "Entrega realizada sin inconvenientes", resultado.getObservaciones());
    verify(envioRepo, times(wantedNumberOfInvocations: 1)).save(envioEnTransito);
}
```

illegalStateException si no está en tránsito

```
@Test
@DisplayName("confirmarEntrega - debe lanzar IllegalStateException si no está EN_TRANSITO")
void testConfirmarEntrega_EstadoInvalido() {
    // envioPlanificado está PLANIFICADO, no se puede confirmar entrega
    ActualizarEstadoRequest request = new ActualizarEstadoRequest();
    request.setObservaciones(observaciones: "Intento inválido");

    when(envioRepo.findByIdWithDetalle(id: 1L)).thenReturn(Optional.of(envioPlanificado));

    assertThrows(expectedType: IllegalStateException.class,
        () -> logisticaService.confirmarEntrega(envioId: 1L, request));

    verify(envioRepo, never()).save(any());
}
```

Cancelar y devolver stock al inventario

```
@Test
@DisplayName("cancelarEnvio - debe cancelar y devolver el stock al inventario")
void testCancelarEnvio_Exitoso() {
    ActualizarEstadoRequest request = new ActualizarEstadoRequest();
    request.setObservaciones(observaciones: "Cancelado por falta de transporte");

    BigDecimal stockAntes = itemAlimentos.getCantidadDisponible(); // 200 kg

    when(envioRepo.findByIdWithDetalle(id: 1L)).thenReturn(Optional.of(envioPlanificado));
    // El envío tiene 1 detalle: 30 kg de ALIMENTOS → deben devolverse al inventario
    when(inventarioRepo.findByIdAndTipoRecurso(centroAcopioId: 1L, TipoRecurso.ALIMENTOS))
        .thenReturn(Optional.of(itemAlimentos));
    when(inventarioRepo.save(any(type: ItemInventario.class))).thenReturn(itemAlimentos);
    when(envioRepo.save(any(type: Envio.class))).thenReturn(invoc -> invoc.getArgument(index: 0));

    EnvioResponse resultado = logisticaService.cancelarEnvio(envioId: 1L, request);

    assertNotNull(resultado);
    assertEquals(expected: "CANCELADO", resultado.getEstado());
    assertEquals(expected: "Cancelado por falta de transporte", resultado.getObservaciones());
    // Stock devuelto: 200 + 30 = 230
    assertEquals(stockAntes.add(new BigDecimal("30.00")),
        itemAlimentos.getCantidadDisponible());
    verify(inventarioRepo, times(wantedNumberOfInvocations: 1)).save(itemAlimentos);
    verify(envioRepo, times(wantedNumberOfInvocations: 1)).save(envioPlanificado);
}
```

Lanza IllegalStateException si el envío ya fue entregado

```
@Test
@DisplayName("cancelarEnvio - debe lanzar IllegalStateException si el envío ya fue entregado")
void testCancelarEnvio_YaEntregado() {
    Envio envioEntregado = new Envio();
    envioEntregado.setId(id: 3L);
    envioEntregado.setEstado(EstadoEnvio.ENTREGADO);
    envioEntregado.setCentroOrigen(centroActivo);
    envioEntregado.setDetalle(new ArrayList<>());
    envioEntregado.setDestinoDescripcion(destinoDescripcion: "x");
    envioEntregado.setDestinoDireccion(destinoDireccion: "x");
    envioEntregado.setDestinoRegion(destinoRegion: "x");
    envioEntregado.setDestinoComuna(destinoComuna: "x");

    ActualizarEstadoRequest request = new ActualizarEstadoRequest();
    request.setObservaciones(observaciones: "Intento de cancelación inválido");

    when(envioRepo.findByIdWithDetalle(id: 3L)).thenReturn(Optional.of(envioEntregado));

    assertThrows(expectedType: IllegalStateException.class,
        () -> logisticaService.cancelarEnvio(envioId: 3L, request));

    verify(envioRepo, never()).save(any());
}
```

Retorna envío con el número indicado

```
@Test
@DisplayName("buscarPorNumeroSeguimiento - debe retornar el envío con el número indicado")
void testBuscarPorNumeroSeguimiento_Exitoso() {
    when(envioRepo.findByNumeroSeguimiento(numeroSeguimiento: "DON-A3F92C01"))
        .thenReturn(Optional.of(envioPlanificado));

    EnvioResponse resultado =
        logisticaService.buscarPorNumeroSeguimiento(numeroSeguimiento: "DON-A3F92C01");

    assertNotNull(resultado);
    assertEquals(expected: "DON-A3F92C01", resultado.getNumeroSeguimiento());
    assertEquals(expected: "PLANIFICADO", resultado.getEstado());
}
```

Buscar por número de seguimiento

```
@Test
@DisplayName("buscarPorNumeroSeguimiento - debe lanzar IllegalArgumentException si no existe")
void testBuscarPorNumeroSeguimiento_NoEncontrado() {
    when(envioRepo.findByNumeroSeguimiento(numeroSeguimiento: "DON-XXXXXXXX"))
        .thenReturn(Optional.empty());

    assertThrows(expectedType: IllegalArgumentException.class,
        () -> logisticaService.buscarPorNumeroSeguimiento(numeroSeguimiento: "DON-XXXXXXXX"));
}
```

Listar envío

```
@Test
@DisplayName("listarEnviosPorCentro - debe retornar página de envíos del centro indicado")
void testListarEnviosPorCentro_Exitoso() {
    Pageable pageable = PageRequest.of(pageNumber: 0, pageSize: 10);
    Page<Envio> pagina =
        new PageImpl<>(Arrays.asList(envioPlanificado, envioEnTransito));

    when(envioRepo.findByCentroOrigenIdOrderByFechaCreacionDesc(centroOrigenId: 1L, pageable))
        .thenReturn(pagina);

    Page<EnvioResponse> resultado = logisticaService.listarEnviosPorCentro(centroId: 1L, pageable);

    assertNotNull(resultado);
    assertEquals(expected: 2, resultado.getTotalElements());
    verify(envioRepo, times(wantedNumberOfInvocations: 1))
        .findByCentroOrigenIdOrderByFechaCreacionDesc(centroOrigenId: 1L, pageable);
}
```

Retorna los envíos paginados

```
@Test
@DisplayName("listarTodosLosEnvios - debe retornar todos los envíos paginados")
void testListarTodosLosEnvios_Exitoso() {
    Pageable pageable = PageRequest.of(pageNumber: 0, pageSize: 10);
    Page<Envio> pagina =
        new PageImpl<>(Arrays.asList(envioPlanificado, envioEnTransito));

    when(envioRepo.findAllByOrderByFechaCreacionDesc(pageable)).thenReturn(pagina);

    Page<EnvioResponse> resultado = logisticaService.listarTodosLosEnvios(pageable);

    assertNotNull(resultado);
    assertEquals(expected: 2, resultado.getTotalElements());
    verify(envioRepo, times(wantedNumberOfInvocations: 1)).findAllByOrderByFechaCreacionDesc(pageable);
}
```

Retorna páginas vacías si no hay envíos

```
@Test
@DisplayName("listarTodosLosEnvios - debe retornar página vacía si no hay envíos")
void testListarTodosLosEnvios_Vacio() {
    Pageable pageable = PageRequest.of(pageNumber: 0, pageSize: 10);
    when(envioRepo.findAllByOrderByFechaCreacionDesc(pageable))
        .thenReturn(Page.empty());

    Page<EnvioResponse> resultado = logisticaService.listarTodosLosEnvios(pageable);

    assertNotNull(resultado);
    assertEquals(expected: 0, resultado.getTotalElements());
}
```

Conclusión

El diseño e implementación de la plataforma "Donaton" demuestra cómo la integración de tecnologías modernas de desarrollo y despliegue puede transformar procesos manuales y aislados en una operación logística centralizada, transparente y de alta eficiencia ante situaciones de emergencia.

A lo largo de este proyecto, la adopción de una arquitectura basada en **microservicios independientes** (Donación, Necesidades y Logística) garantizó que el sistema cumpla con una de sus necesidades más críticas: la alta disponibilidad. Al segmentar las responsabilidades y permitir la comunicación reactiva no bloqueante mediante **Spring WebClient**, se asegura que la plataforma sea capaz de absorber un alto volumen de peticiones concurrentes durante un desastre natural sin colapsar.

Asimismo, las decisiones en infraestructura y calidad de software robustecieron la solución:

- **Escalabilidad y Resiliencia:** El uso de contenedores con **Docker** y su orquestación mediante **Kubernetes (Amazon EKS)**, respaldados por la nube de **AWS**, dotan al sistema de propiedades autocurativas y escalabilidad horizontal automática, permitiendo responder con inmediatez al incremento abrupto de tráfico en crisis sociales.
- **Calidad y Confianza:** El desarrollo de una rigurosa batería de **pruebas unitarias utilizando JUnit 5 y Mockito, SonarQube** permitió verificar de forma aislada e inline las reglas de negocio críticas (como la gestión estricta de inventarios y validación de donantes), alcanzando una cobertura superior al 90% y garantizando la estabilidad del código antes de ser integrado.
- **Trabajo Estructurado:** La metodología de control de versiones con **Git y GitHub**, basada en una estrategia de ramificación limpia, permitió un flujo de trabajo ordenado, mitigando conflictos en el código principal (**main**) y asegurando la trazabilidad del desarrollo.

En definitiva, la arquitectura propuesta no solo responde con solidez a los exigentes requerimientos técnicos de la asignatura, sino que se alinea por completo con la misión humanitaria de la organización, sentando una base tecnológica segura, sostenible y óptima para salvar vidas a través de una correcta coordinación de la ayuda.

Bibliografía

Amazon Web Services. (2025). *AWS Architecture Center: Microservices on Kubernetes*. <https://aws.amazon.com/architecture>

Duoc UC. (s.f.). *Documentación de Caso Semestral "Donaton"*.

Google. (2026). *Gemini IA (Modelo de lenguaje extenso)*.
<https://gemini.google.com>

Refactoring.Guru. (2024). *Repository and MVC Patterns*.
<https://refactoring.guru>

Spring.io. (2024). *Spring Cloud OpenFeign Documentation*.
<https://spring.io/projects/spring-cloud-openfeign>